

Create a one-dimensional Array of strings and one of integers. Ask the user to select an index of the Array and then display the string at that index on the screen.

Add in a message that displays when the user selects an index that doesn't exist.

Create a list of strings. Ask the user to select an index of the list and then display the content at that index on the screen.

A one-dimensional array of strings.

Ask the user to input some text.

A loop that iterates through each string in the array and adds the user's text input to the end of each string. This step will not output anything to the console, but will update each array element by appending the user's text.

Then create a second loop that prints off each string in the array one at a time.

Save and execute your code.

An infinite loop.

Save your code.

Fix the infinite loop so that it will execute properly.

Add the following to your console app and perform these actions:

A list of strings where each item in the list is unique.

Ask the user to input text to search for in the list.

A loop that iterates through the list and then displays the index of the list item that contains matching text on the screen.

Add code to check if the user put in text that isn't on the list and, if they did, tell the user their input is not on the list.

Add code that stops the loop from executing once a match has been found.

Add the following to your console app and perform these actions:

A list of strings that has at least two identical strings in the list.

Ask the user to select text to search for in the list.

Create a loop that iterates through the list and then displays the indices of the items matching the user-selected text. Ensure to remove any break statements that may prevent your code from returning multiple matches.

Add code to the loop to check if the user put in text that isn't on the list and, if they did, tells the user their input is not on the list.

(You do *not* need to provide any additional chances for the user to enter text.)

Save and execute your code.

Add the following to your console app and perform these actions:

Create a list of strings that has at least two identical strings in the list.

Create a foreach loop that evaluates each item in the list, and displays a message showing the string and whether or not it has already appeared in the list. For example, if you had a list of letters: {"A", "B", "C", "D", "C"}, you could have the following print to the screen:

```
A - this item is unique
B - this item is unique
C - this item is unique
D - this item is unique
C - this item is a duplicate
```

Save and execute your code.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload all the code you wrote during every step of this assignment to your GitHub account.

Perform these actions and create a console app that includes the following:

Create a list of integers. Ask the user for a number to divide each number in the list by. Write a loop that takes each integer in the list, divides it by the number the user entered, and displays the result to the screen.

Run the code, entering in non-zero numbers as the user. Look at the displayed results.

Run the code again, entering in zero as the number to divide by. Note any error messages you get.

Run the code once again, entering in a string as the number to divide by. Note any error messages you get.

Now put the loop in a try/catch block. Below and outside of the try/catch block, make the program print a message to the display to let you know the program has emerged from the try/catch block and continued on with program execution. In the catch block, display the error message to the screen. Then try various combinations of user input: valid numbers, zero and a string. Ensure the proper error messages display on the screen, and that the code after the try/catch block gets executed.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Create a class. In that class, create three methods, each of which will take one integer parameter in and return an integer. The methods should do some math operation on the received parameter. Put this class in a separate .cs file in the application. In the Main() program, ask the user what number they want to do the math operations on.

Call each method in turn, passing the user input to the method. Display the returned integer to the screen.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Create a class. In that class, create a method that will take in an integer, create a math operation for this integer (addition, subtraction, etc.), then return the answer as an integer.

In the Main() method of the console app, instantiate the class and call the one method, passing in an integer. Display the result to the screen.

Add a second method to the class with the same name that will take in a decimal, create a different math operation for it, then return the answer as an integer.

In the Main() method of the console app, instantiate the class and call the second method, passing in a decimal. Display the result to the screen.

Add a third method to the class, with the same name, that will take in a string, convert it to an integer if possible, do a different math operation to it, then return the answer as an integer.

In the Main() method of the console app, instantiate the class and call the third method, passing in a string that equates to an integer. Display the result to the screen.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Create a class. In that class, create a method that takes two integers as parameters. Make one of them optional. Have the method do a math operation and return an integer result.

In the Main() method of the console app, instantiate the class.

Ask the user to input two numbers, one at a time. Let them know they need not enter anything for the second number.

Call the method in the class, passing in the one or two numbers entered.

Try various combinations of numbers on the code, including having no second number.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Create a class. In that class, create a void method that takes two integers as parameters. Have the method do a math operation on the first integer and display the second integer to the screen.

In the Main() method of the console app, instantiate the class.

Call the method in the class, passing in two numbers.

Call the method in the class, specifying the parameters by name.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Create a class. In that class, create a void method that outputs an integer. Have the method divide the data passed to it by 2.

In the Main() method, instantiate that class.

Have the user enter a number. Call the method on that number. Display the output to the screen. It should be the entered number, divided by two.

Create a method with output parameters.

Overload a method.

Declare a class to be static.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Create a class called Person and give it two properties, each of data type string. One called FirstName, the other LastName.

Give this class a void method called SayName() that takes no parameters and simply writes the person's full name to the console in the format of: "Name: [full name]".

Create another class called Employee and have it inherit from the Person class. Give the Employee class a property called Id and have it be of data type int.

Inside of the Main method, instantiate and initialize an Employee object with a first name of "Sample" and a last name of "Student".

Call the superclass method SayName() on the Employee object.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Create an abstract class called Person with two properties: string firstName and string lastName.

Give it the method SayName().

Create another class called Employee and have it inherit from the Person class.

Implement the SayName() method inside of the Employee class.

Inside the Main() method, instantiate an Employee object with firstName "Sample" and lastName "Student". Call the SayName() method on the object.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Create an interface called IQuittable and have it define a void method called Quit().

Have your Employee class from the previous drill inherit that interface and implement the Quit() method in any way you choose.

Use polymorphism to create an object of type IQuittable and call the Quit() method on it. Hint: an object can be of an interface type if it implements that specific interface.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Create an Employee class with Id, FirstName and LastName properties.

In the Employee class, overload the “==” operator so it checks if two Employee objects are equal by comparing their Id property. Remember that comparison operators must be overloaded in pairs.

In the "Program.cs" file, instantiate two objects of the Employee class and assign values to their properties. Then compare the two Employee objects using the newly overloaded operators and display the results.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Make the Employee class take a generic type parameter.

Add a property to the Employee class called “things” and have its data type be a generic list matching the generic type of the class.

Instantiate an Employee object with type “string” as its generic parameter. Assign a list of strings as the property value of Things.

Instantiate an Employee object with type “int” as its generic parameter. Assign a list of integers as the property value of Things.

Create a loop that prints all of the Things to the Console.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

NOTE: This assignment is a challenge. To complete this challenge, you will need to do some online research on how to parse enums.

Perform these actions and create a console app that includes the following:

Create an enum for the days of the week.

Prompt the user to enter the current day of the week.

Assign the value to a variable of that enum data type you just created.

Wrap the above statement in a try/catch block and have it print "Please enter an actual day of the week." to the console if an error occurs.

Perform these actions and create a console app that includes the following:

Create a struct called Number and give it the property "Amount" and have it be of data type decimal.

In the Main() method, create an object of data type Number and assign an amount to it.

Print this amount to the console.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Create an Employee class with the following properties:

- a. Id
- b. First Name
- c. Last Name

In the Main() method, create a list of at least 10 employees. At least two employees should have the first name “Joe”.

Using a foreach loop, create a new list of all employees with the first name “Joe”. In your comparison statement, remember to reference the property of the object you are checking.

Perform the same action again, but this time with a lambda expression.

Using a lambda expression, make a list of all employees with an Id number greater than 5.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform the following actions:

Set a breakpoint on the first line of the Main method in the Program Class of the TwentyOne console application.

Start the program and use F11 or the Step Into button of Visual Studio's debugger to go step-by-step through the entire program. Do this until you understand what every part of the program is doing (research online, as needed).

After you have done that, figure out the answers to these questions:

What is this line of code doing (in TwentyOneRules class)?:

```
int playerScore = playerResults.Where(x => x < 22).Max();
```

What is this block of code doing (in the Main method)?:

```
while (player.isActivelyPlaying && player.Balance > 0)
{
    {
        game.Play();
    }
}
```

Perform these actions and create a console app that includes the following:

Asks the user for a number.

Logs that number to a text file.

Prints the text file back to the user.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Prints the current date and time to the console.

Asks the user for a number.

Prints to the console the exact time it will be in X hours, X being the number the user entered in step 2.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Create a const variable.

Create a variable using the keyword “var.”

Chain two constructors together.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

Perform these actions and create a console app that includes the following:

Ask the user for their age.

Display the year the user was born.

Exceptions must be handled using “try/catch.”

Display appropriate error messages if the user enters zero or negative numbers.

Display a general message if an exception was caused by anything else.

Add comments to each line or block of your code to explain what it does exactly, so that another developer could read and understand your code.

Upload your code to GitHub.

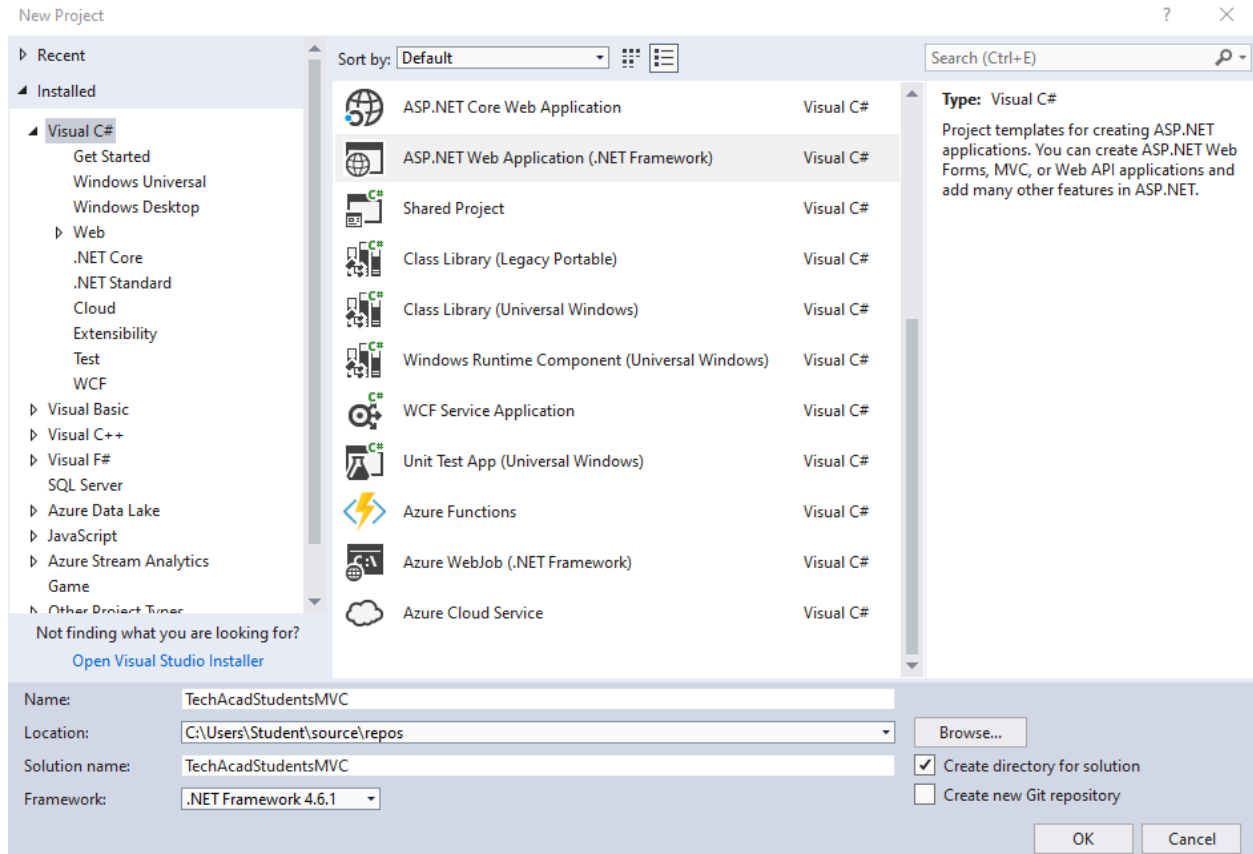
C# and .Net Major Projects

In this assignment, you will build a basic MVC application.

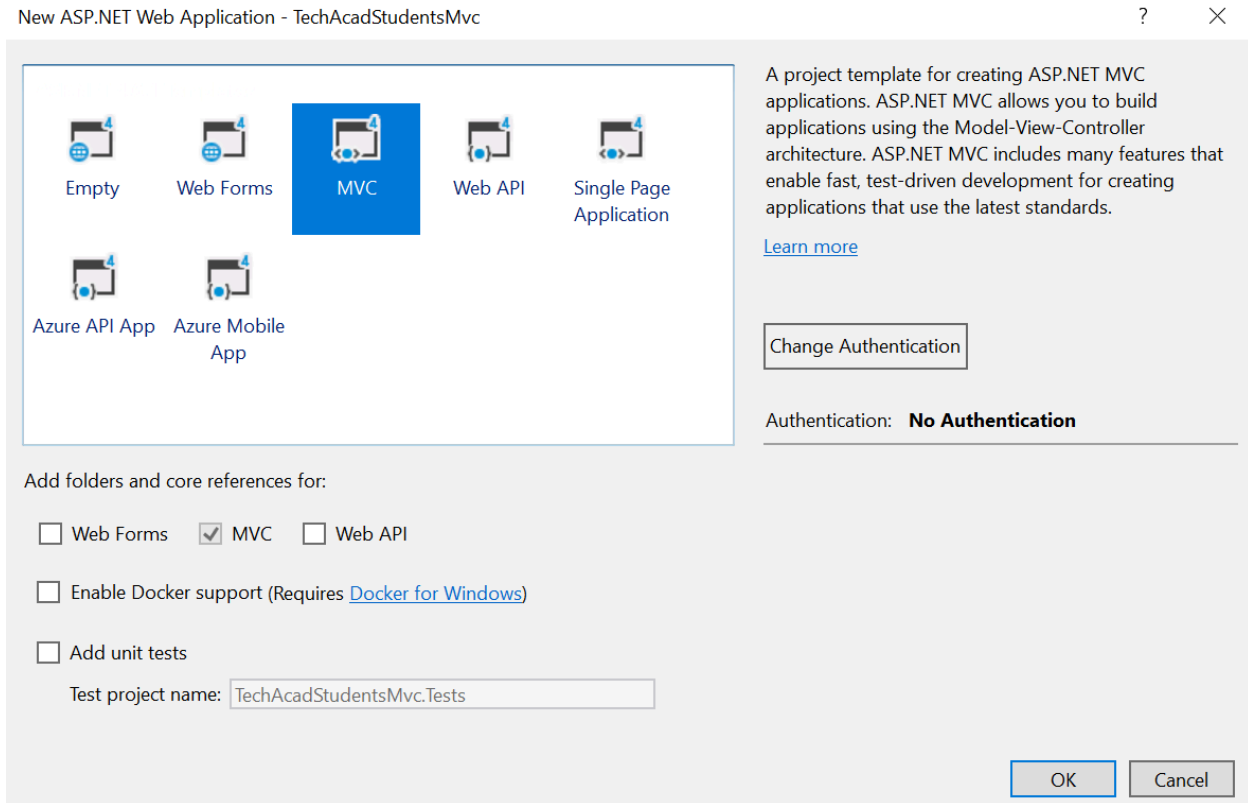
Perform these actions:

Click File > New Project > Visual C# > Web > ASP.NET Web Application (.NET Framework).

Name your project TechAcadStudentsMVC.



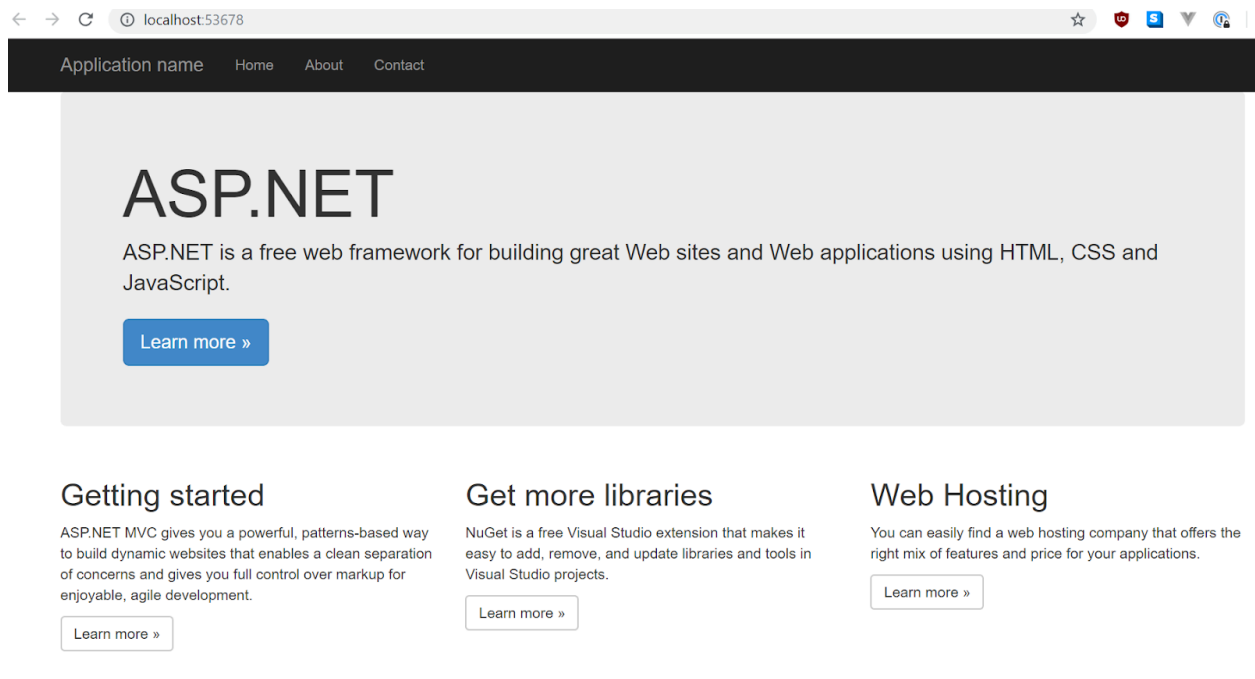
On the next screen, select MVC, then click OK.



After you click OK, allow Visual Studio time to “spin up” the project. The advantage and power of Visual Studio is that it provides you a basic functioning application without any coding. Once your application is loaded, click the green play button in the top middle of Visual Studio or press F5. This “starts debugging” your application, which simply means it’s running while connected to Visual Studio. So any time the program hits an error or a breakpoint, Visual Studio shows you exactly where in the code the bug or breakpoint occurred.

Study these steps and perform any directed actions:

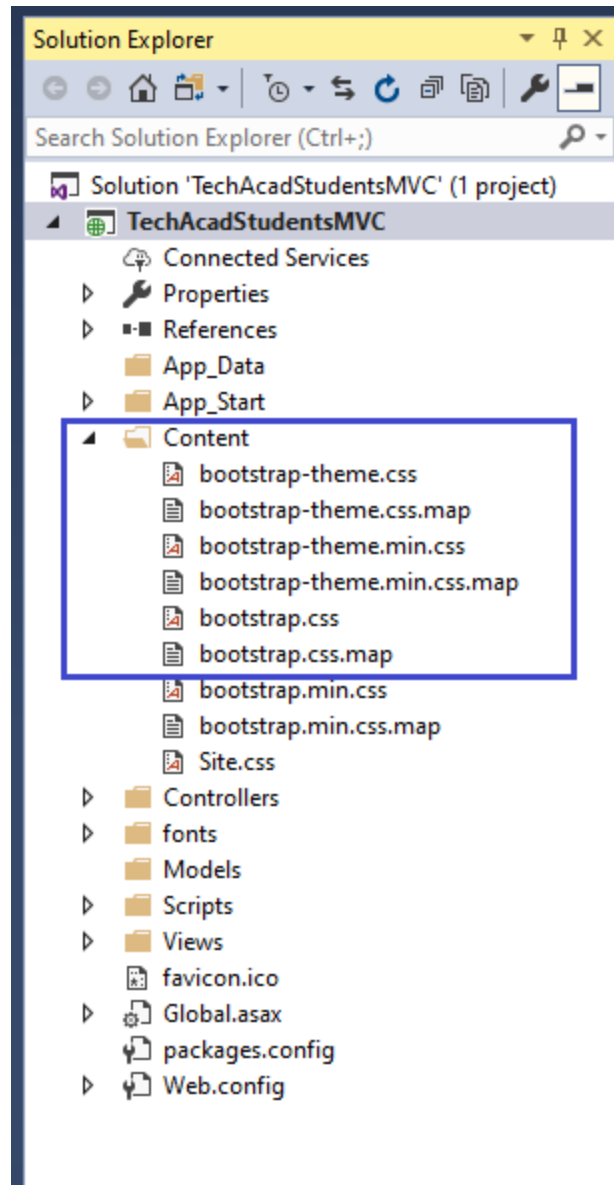
View the application.



Visual Studio is running the application using a stripped down version of Internet Information Services, called IIS Express.

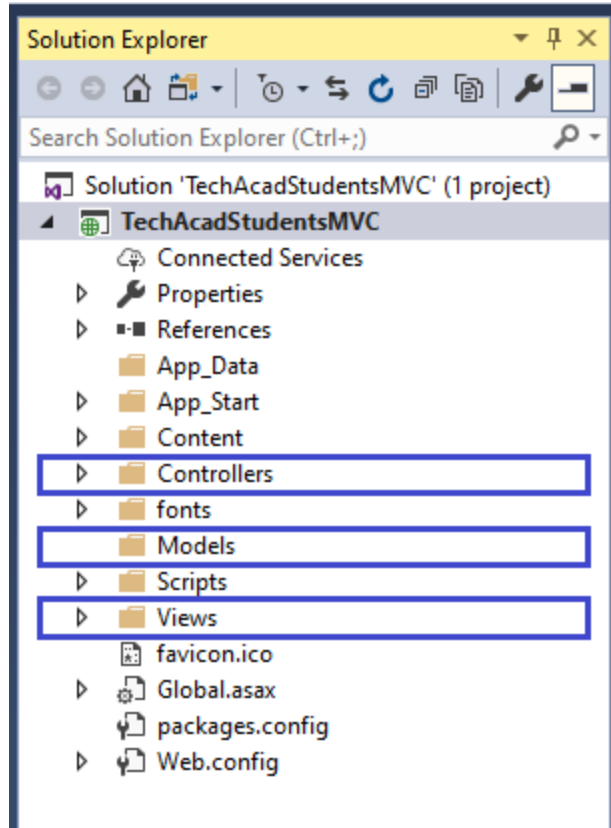
Take a minute to explore the application. Notice that there is already a Home, About and Contact page. The application is also mobile-friendly.

Back in Visual Studio on the right hand side in Solution Explorer, locate all the files for your project (*TechAcadStudentMvc*). You'll notice a file called *bootstrap.css* in the Content folder.



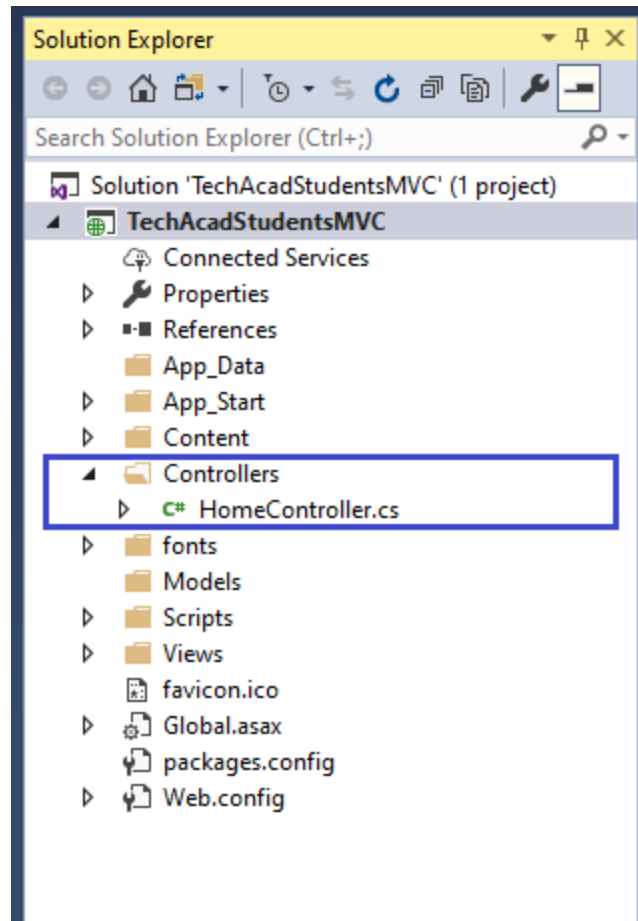
You don't have to use Bootstrap, but as you can see, it is included by default in all new MVC projects.

Notice that your project contains folders for each Models, Views and Controllers.



You can think about MVC in the simplest possible manner: It's a way of organizing the important parts of your code into 3 folders. The files in each folder do very specific things. Let's start with the Controllers folder. Study these steps and perform any directed actions:

Inside the Controllers folder, you'll see that it contains one file: HomeController.cs.



Every file in the Controllers folder is called a “controller.” A controller is a Class whose job is to figure out what to show the user.

Its naming always follows the same format:

[thing_being_controlled][Controller].cs

Examples:

HomeController.cs

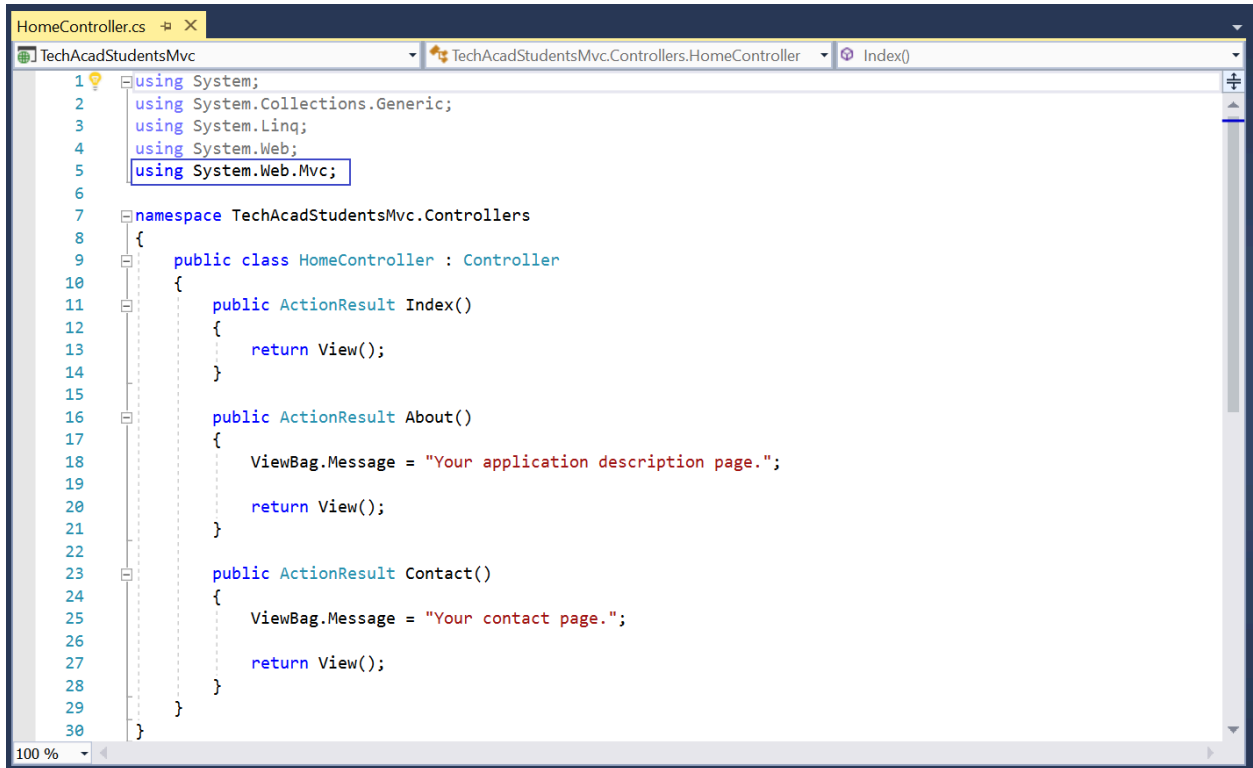
FileController.cs

JobController.cs

FormController.cs

PersonnelController.cs

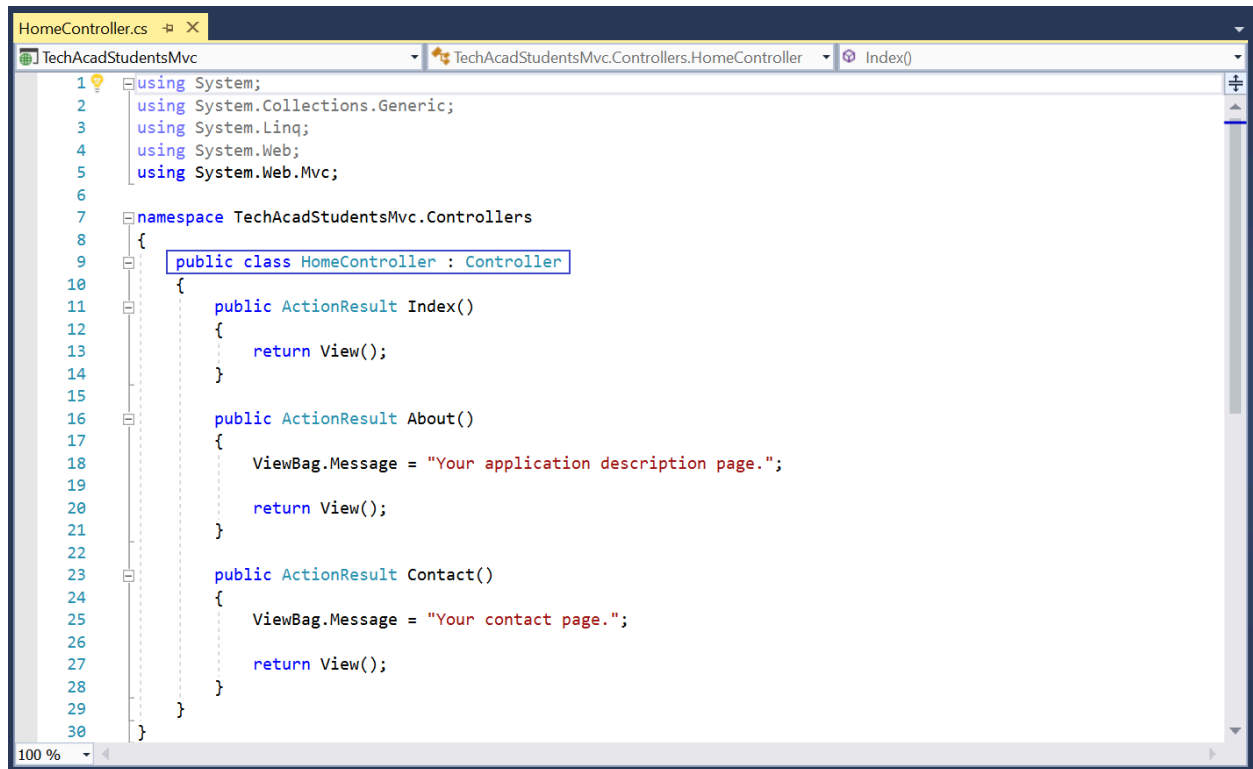
Open the HomeController.cs file:



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5 using System.Web.Mvc;
6
7 namespace TechAcadStudentsMvc.Controllers
8 {
9     public class HomeController : Controller
10    {
11        public ActionResult Index()
12        {
13            return View();
14        }
15
16        public ActionResult About()
17        {
18            ViewBag.Message = "Your application description page.";
19
20            return View();
21        }
22
23        public ActionResult Contact()
24        {
25            ViewBag.Message = "Your contact page.";
26
27            return View();
28        }
29    }
30 }
```

Notice that it is using the System.Web.Mvc namespace. This is just one of the many libraries included in .NET's powerful library of code called the Framework Class Library.

Now notice that the class name is called HomeController (same as the file name) and inherits from a class called Controller.



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5 using System.Web.Mvc;
6
7 namespace TechAcadStudentsMvc.Controllers
8 {
9     public class HomeController : Controller
10    {
11        public ActionResult Index()
12        {
13            return View();
14        }
15
16        public ActionResult About()
17        {
18            ViewBag.Message = "Your application description page.";
19
20            return View();
21        }
22
23        public ActionResult Contact()
24        {
25            ViewBag.Message = "Your contact page.";
26
27            return View();
28        }
29    }
30 }
```

The Controller class provides our HomeController class with the method View() and the pseudo-dictionary called ViewBag.

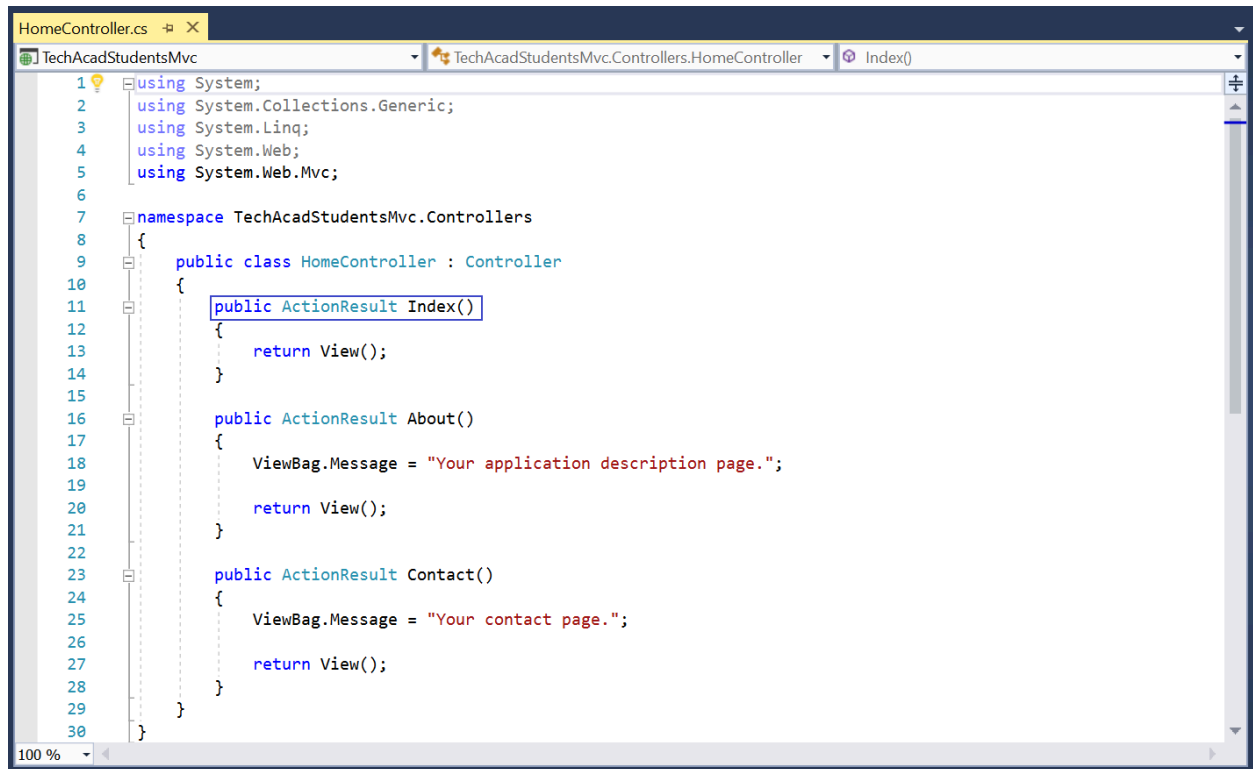
Watch what happens when we stop HomeController from inheriting from Controller:

```
4 using System.Web;
5 using System.Web.Mvc;
6
7 namespace TechAcadStudentsMvc.Controllers
8 {
9     public class HomeController
10     {
11         public ActionResult Index()
12         {
13             return View();
14         }
15
16         public ActionResult About()
17         {
18             ViewBag.Message = "Your application description page.";
19             return View();
20         }
21
22         public ActionResult Contact()
23         {
24             ViewBag.Message = "Your contact page.";
25             return View();
26         }
27     }
28 }
29
30
```

ViewBag and View() are no longer recognized because they were defined in the Controller class (Now be sure to put the Controller class back).

Study these steps and perform any directed actions:

Let's look at this first method called Index(). Analyzing its signature, we see that it accepts no parameters and its return type is a data type called ActionResult.

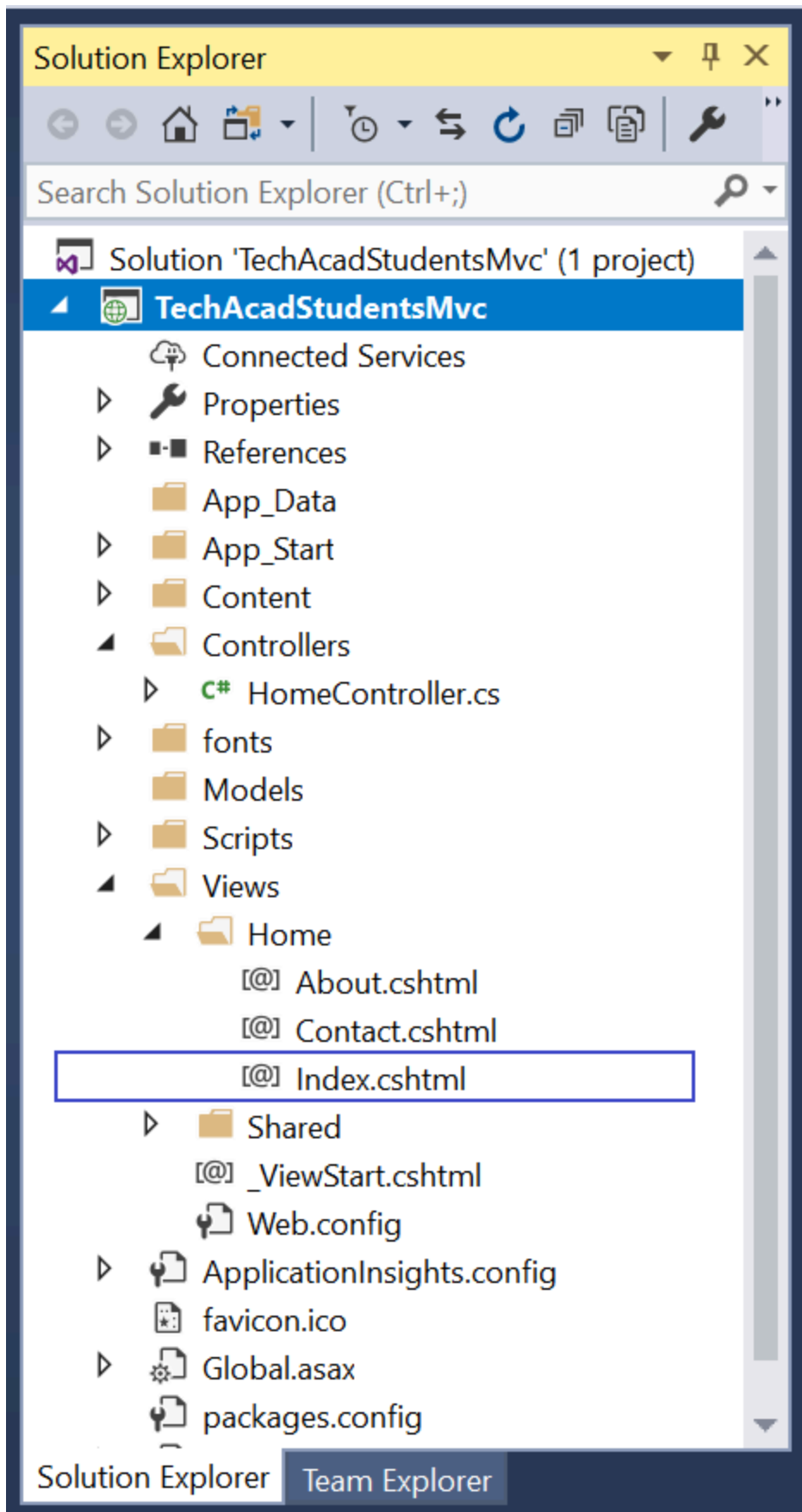


```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5 using System.Web.Mvc;
6
7 namespace TechAcadStudentsMvc.Controllers
8 {
9     public class HomeController : Controller
10    {
11        public ActionResult Index()
12        {
13            return View();
14        }
15
16        public ActionResult About()
17        {
18            ViewBag.Message = "Your application description page.";
19
20            return View();
21        }
22
23        public ActionResult Contact()
24        {
25            ViewBag.Message = "Your contact page.";
26
27            return View();
28        }
29    }
30 }
```

ActionResult is a class defined in the System.Web.Mvc library. For our purposes, you can think of an ActionResult as “rendered HTML,” although it could be as varied as a “processed file” or “JSON-formatted data.” The point is that an action was performed, which action produced a result. That result (or ActionResult), in turn, is being returned by this method.

So we are now returning the result of the View() method (or shall we say... View() action).

What is the result of the View() method? What is that method doing? The answer lies in our Views folder:

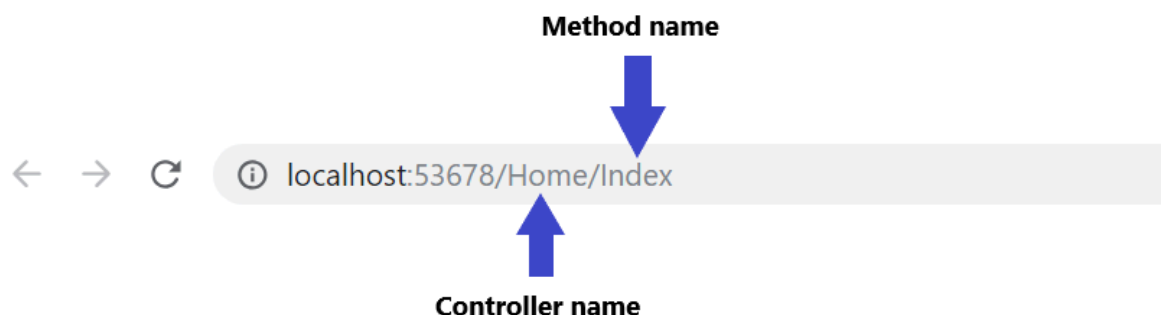


NOTE: The extension .cshtml stands for C Sharp HTML. It is C# code mixed in with HTML. That C# code is processed on the server and then sent to the browser as plain HTML.

The View() method is a very powerful method. By simply looking at the method name it is called from (in our case Index), as well as the Controller Class name (in our case HomeController), it can find and process the appropriate .cshtml file. In this case it is the Index.cshtml file.

How did it know to choose Index.cshtml? This part is very important and most often confused when learning MVC for the first time.

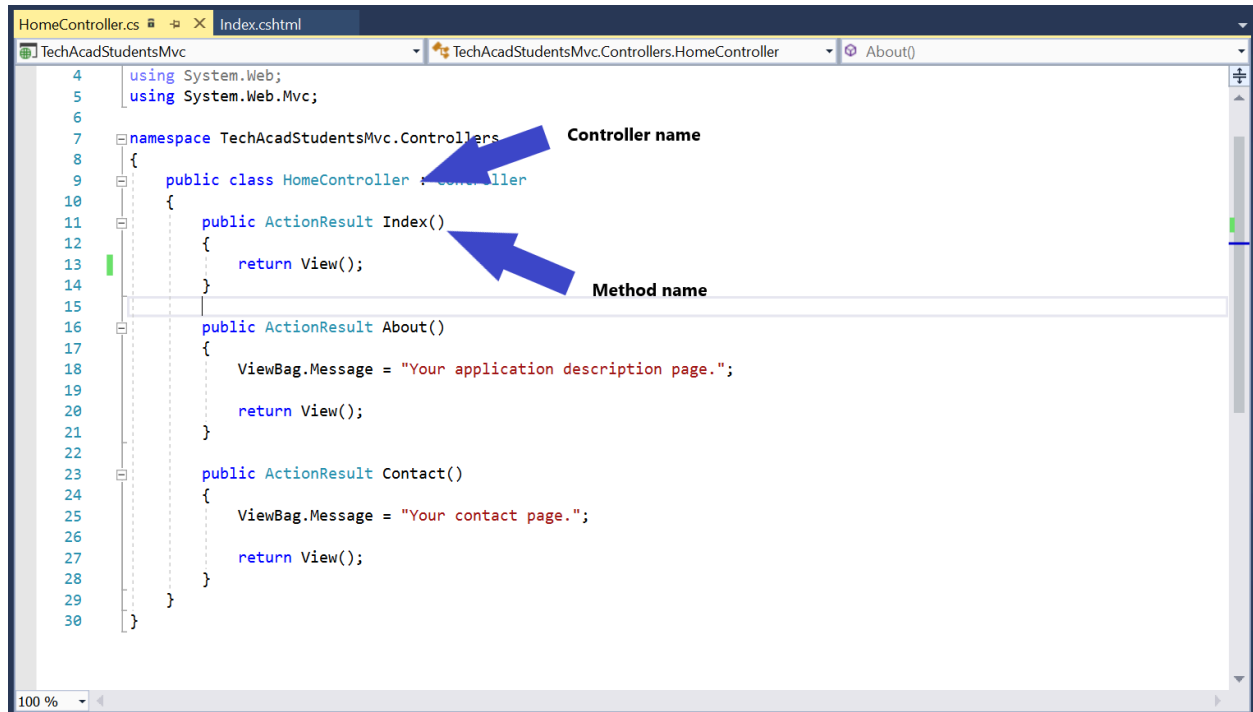
Controllers and views follow a convention, and it starts with the URL.



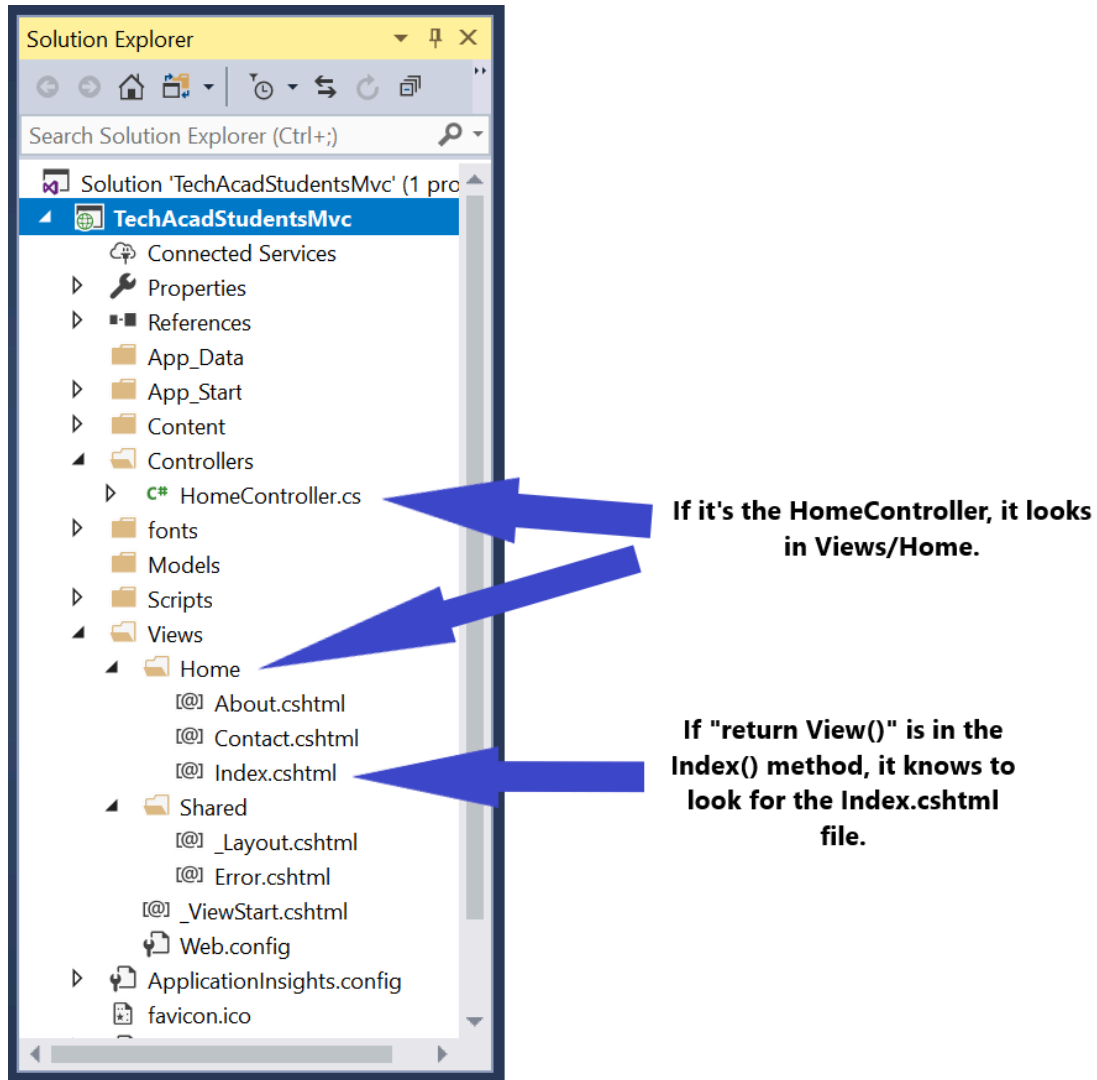
When a user types in this web address, a request goes to the server that is hosting the MVC website. This request then gets routed to the

HomeController. This is why naming Controllers following the convention described earlier is so important.

Once the request hits the HomeController, it then looks for a method named Index().



The `View()` method, in trying to find the appropriate `.cshtml` file to return, follows the same convention:



Here is another example. User navigates to:

/Home/Contact

Controller routed to:

HomeController

Method run:

Contact()

View folder to look in:

Home

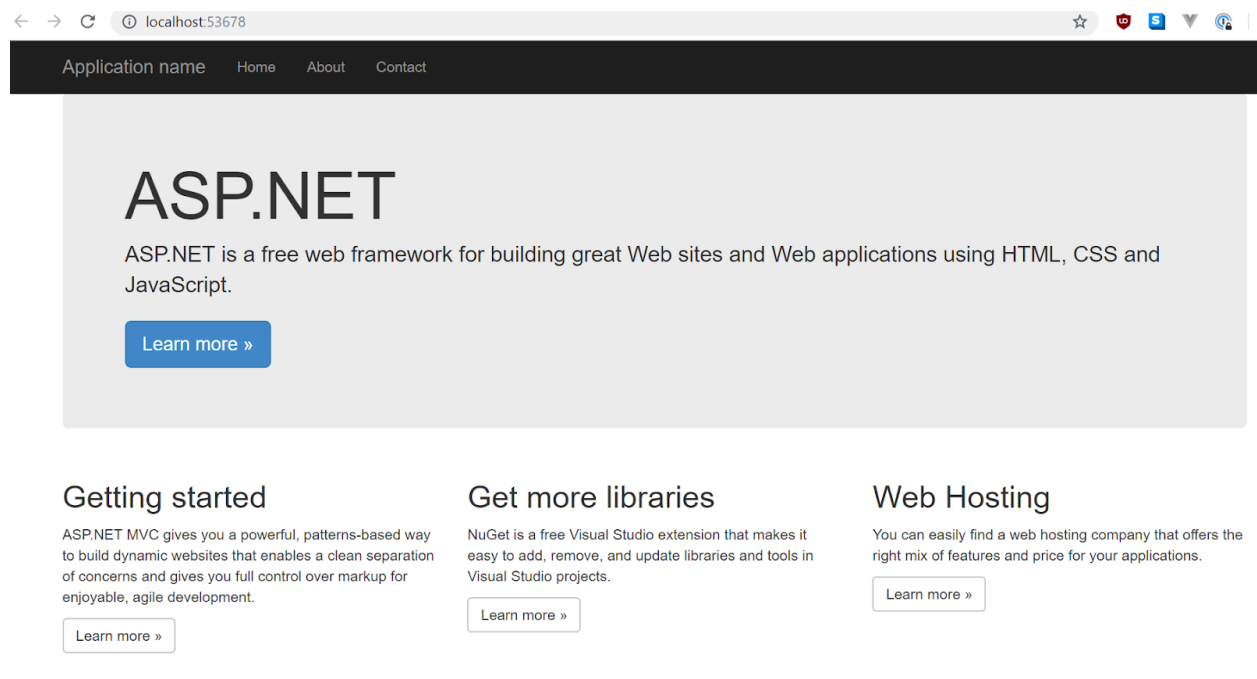
File to return:

Contact.cshtml

Let's see this in action.

Study these steps and perform any directed actions:

Press F5 to start debugging. You should see this screen:



Click on the Contact link at the top.

Contact.

Your contact page.

One Microsoft Way
Redmond, WA 98052-6399
P: 425.555.0100

Support: Support@example.com
Marketing: Marketing@example.com

/Home/Contact

© My ASP.NET Application

Notice that the route in the URL is now /Home/Contact.

Per convention, this request should have gone to the HomeController first. It then goes to the Contact method, which should have then looked in the Views/Home folder for a file called Contact.cshtml.

Let's now open that Contact.cshtml file and see if it is the same one we are looking at.

```
public ActionResult Contact()
{
    ViewBag.Message = "Your contact page.";
    return View();
}
```

```
1 ViewBag.Title = "Contact";
2
3
4 <h2>@ViewBag.Title.</h2>
5 <h3>@ViewBag.Message</h3>
6
7 <address>
8     One Microsoft Way<br />
9     Redmond, WA 98052-6399<br />
10    <abbr title="Phone">P:</abbr>
11    425.555.0100
12 </address>
13
14 <address>
15     <strong>Support:</strong> <a href="mailto:Support@example.com">Support@example.com</a><br />
16     <strong>Marketing:</strong> <a href="mailto:Marketing@example.com">Marketing@example.com</a>
17 </address>
```

Contact.

Your contact page.

One Microsoft Way
Redmond, WA 98052-6399
P: 425.555.0100

Support: Support@example.com
Marketing: Marketing@example.com

Notice that there is some text on the screen that isn't anywhere in the code. This is the basic Layout template and is located in the Views/Shared folder as the `_Layout.cshtml` file.

Also notice that it is making use of ViewBag to store values in the Controller method and display them on the View.

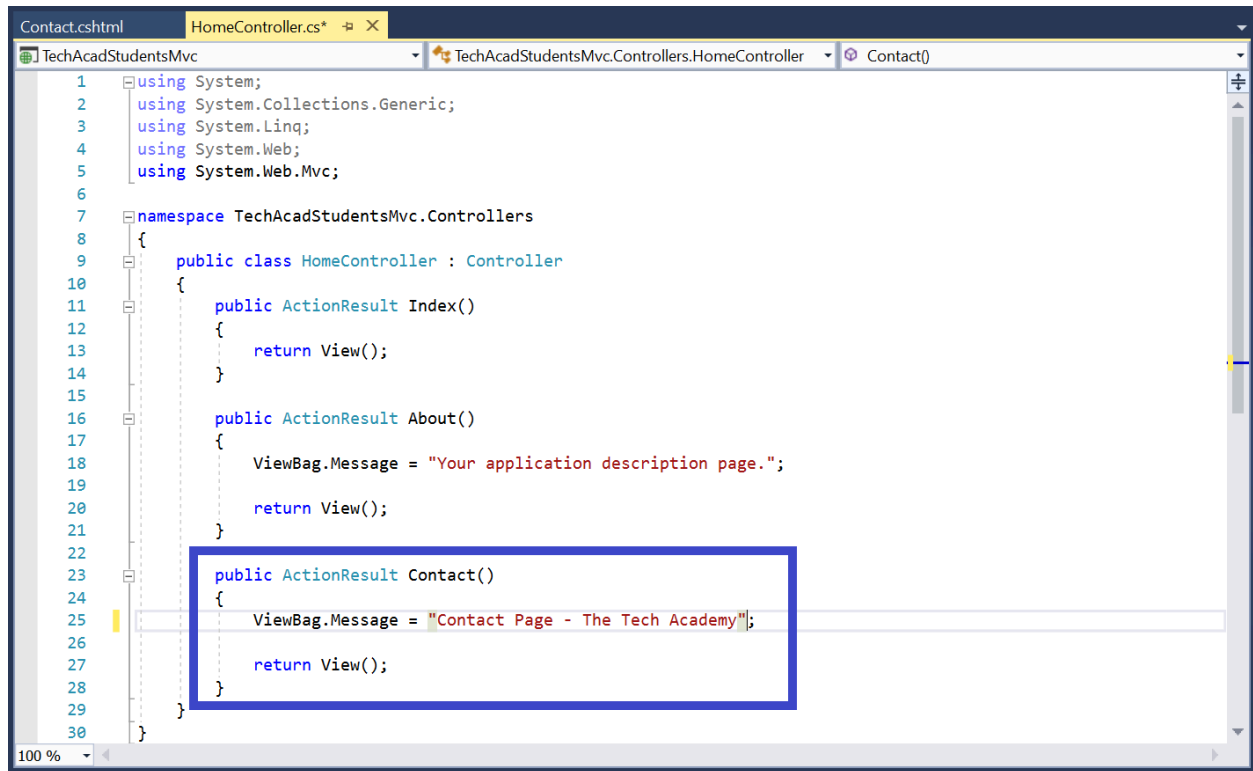
Think of ViewBag like a C# dictionary. It has keys and values. What comes after the dot (".") is the "key," and what comes after the equals sign is the "value." ViewBag is an easy way to store values server side and make them available on the View.

Also, you'll see the Razor syntax in use as the yellow-highlighted @ symbols.

Again, Razor always starts with an @ sign. If the C# code will only be one line, then that's all that is needed. But if it will extend to more than one line, then the code has to also be surrounded by curly braces.

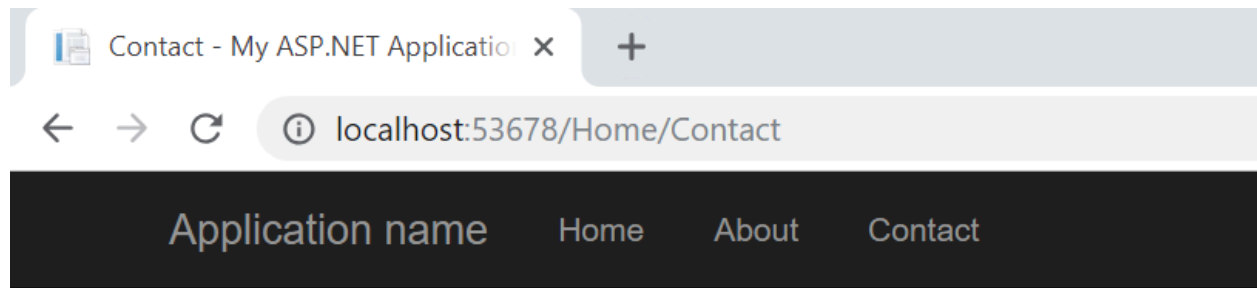
Study these steps and perform any directed actions:

Now let's make some changes to the ViewBag of this Contact method. (You'll need to stop debugging first).



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5 using System.Web.Mvc;
6
7 namespace TechAcadStudentsMvc.Controllers
8 {
9     public class HomeController : Controller
10    {
11        public ActionResult Index()
12        {
13            return View();
14        }
15
16        public ActionResult About()
17        {
18            ViewBag.Message = "Your application description page.";
19
20            return View();
21        }
22
23        public ActionResult Contact()
24        {
25            ViewBag.Message = "Contact Page - The Tech Academy";
26
27            return View();
28        }
29    }
30 }
```

Now let's see the corresponding change when we run the application (press F5 to run):



Contact.

Contact Page - The Tech Academy

One Microsoft Way
Redmond, WA 98052-6399
P: 425.555.0100

Support: Support@example.com

Marketing: Marketing@example.com

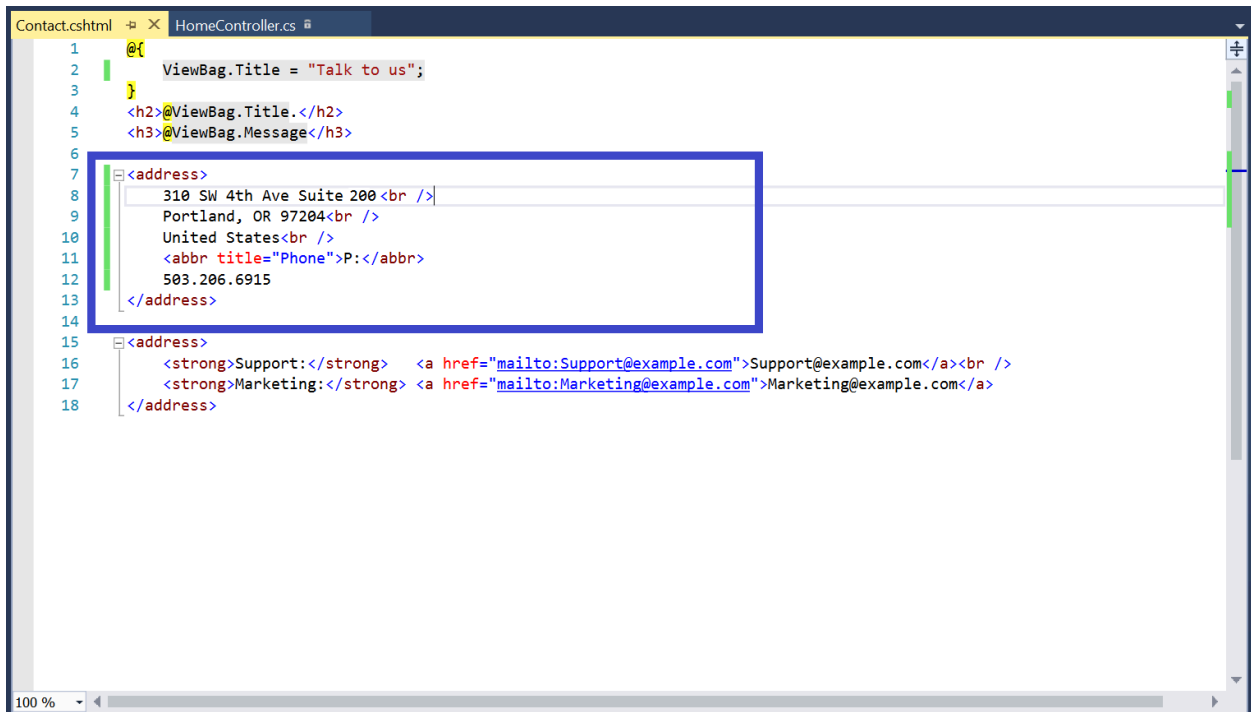
Now let's make a change to the .cshtml file of the application.

When editing .cshtml files, there is no need to stop debugging.

Why is that?

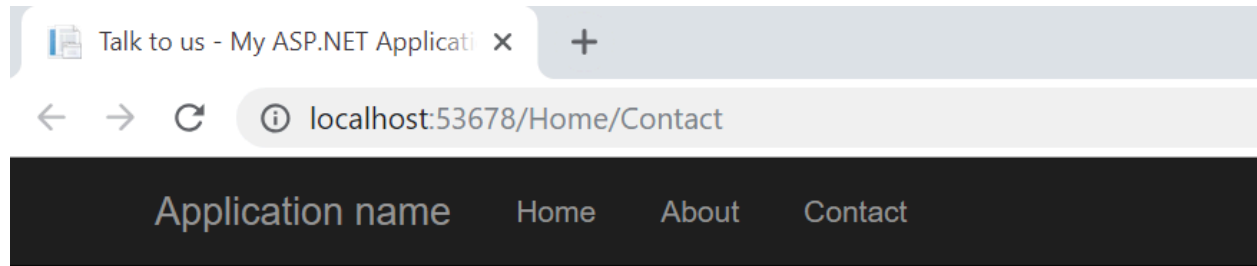
This is because the C# code that lives on the server has to be compiled before it is run. C# inside of HTML is dynamic, so it is rendered by the server as needed.

Let's change the address to reflect that of The Tech Academy.



```
1 @section
2 {
3     ViewBag.Title = "Talk to us";
4 }
5 <h2>@ViewBag.Title.</h2>
6 <h3>@ViewBag.Message</h3>
7
8 <address>
9     310 SW 4th Ave Suite 200 <br />
10    Portland, OR 97204<br />
11    United States<br />
12    <abbr title="Phone">P:</abbr>
13    503.206.6915
14 </address>
15
16 <address>
17     <strong>Support:</strong> <a href="mailto:Support@example.com">Support@example.com</a><br />
18     <strong>Marketing:</strong> <a href="mailto:Marketing@example.com">Marketing@example.com</a>
19 </address>
```

Save the changes and refresh the page in the web browser.



Talk to us.

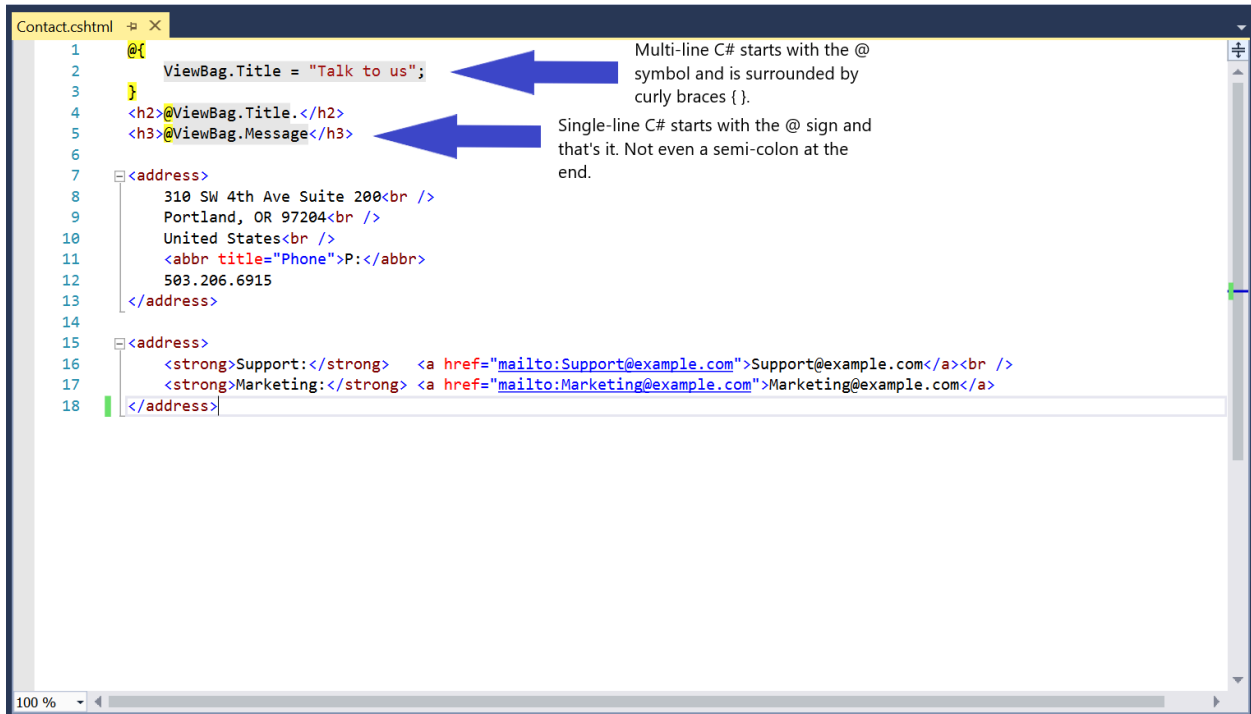
Contact Page - The Tech Academy

310 SW 4th Ave Suite 200
Portland, OR 97204
United States
P: 503.206.6915

Support: Support@example.com

Marketing: Marketing@example.com

So far, we've only changed the HTML of this template. Now let's add some C#. Write this code:



```
1 @  
2 ViewBag.Title = "Talk to us";  
3 }  
4 <h2>@ViewBag.Title.</h2>  
5 <h3>@ViewBag.Message</h3>  
6  
7 <address>  
8     310 SW 4th Ave Suite 200<br />  
9     Portland, OR 97204<br />  
10    United States<br />  
11    <abbr title="Phone">P:</abbr>  
12    503.206.6915  
13 </address>  
14  
15 <address>  
16     <strong>Support:</strong> <a href="mailto:Support@example.com">Support@example.com</a><br />  
17     <strong>Marketing:</strong> <a href="mailto:Marketing@example.com">Marketing@example.com</a>  
18 </address>
```

Multi-line C# starts with the @ symbol and is surrounded by curly braces { }.

Single-line C# starts with the @ sign and that's it. Not even a semi-colon at the end.

Study these steps and perform any directed actions:

Add this C# code:



```
1 @  
2 ViewBag.Title = "Talk to us";  
3 }  
4 <h2>@ViewBag.Title.</h2>  
5 <h3>@ViewBag.Message</h3>  
6  
7 <address>  
8     310 SW 4th Ave Suite 200<br />  
9     Portland, OR 97204<br />  
10    United States<br />  
11    <abbr title="Phone">P:</abbr>  
12    503.206.6915  
13 </address>  
14  
15 <address>  
16     <strong>Support:</strong> <a href="mailto:Support@example.com">Support@example.com</a><br />  
17     <strong>Marketing:</strong> <a href="mailto:Marketing@example.com">Marketing@example.com</a>  
18 </address>  
19  
20 <small>  
21     Contact Information last updated: @DateTime.Now  
22 </small>
```

Refresh the web browser and see the result.

That's a View; we used C# in the View and we used Razor syntax.

Let's add another method to our HomeController and see what else we can do with MVC.

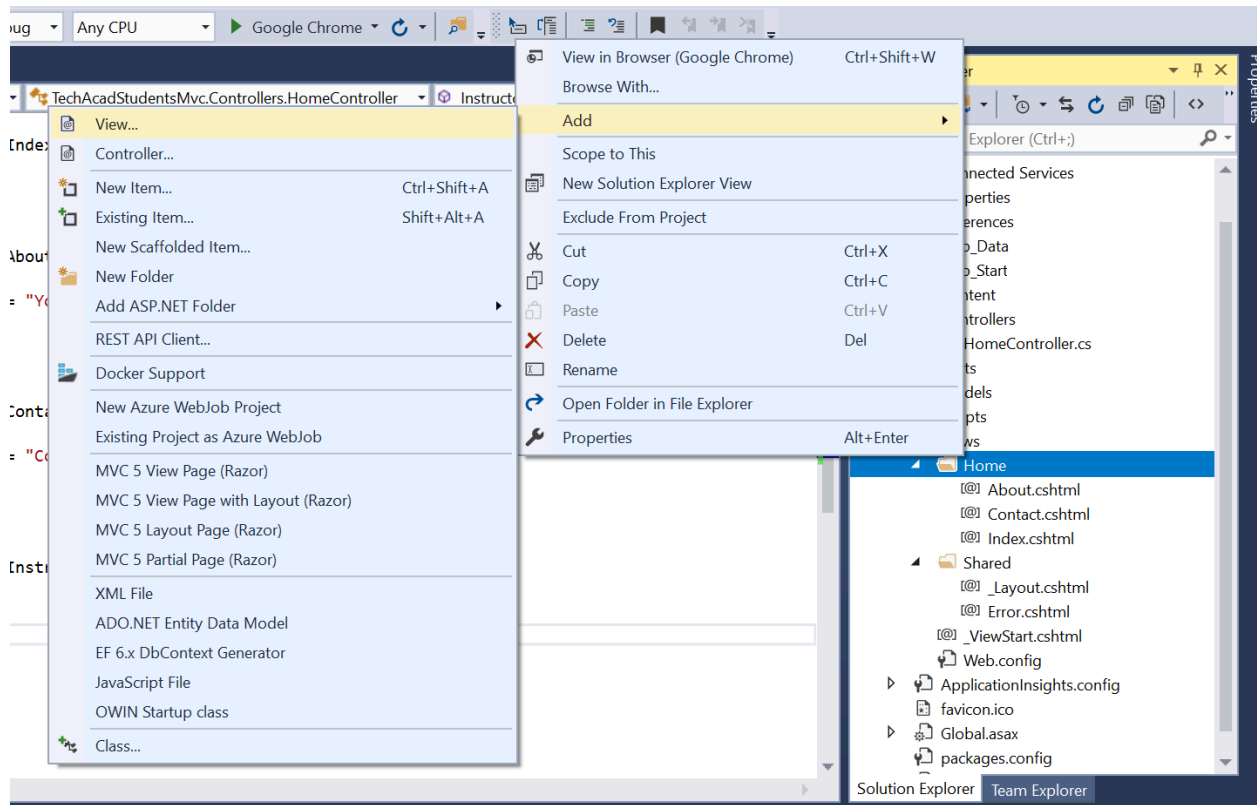
Create a method called `Instructors()` that takes no parameters and returns an `ActionResult`.

Then inside that method, just have it return `View()`.

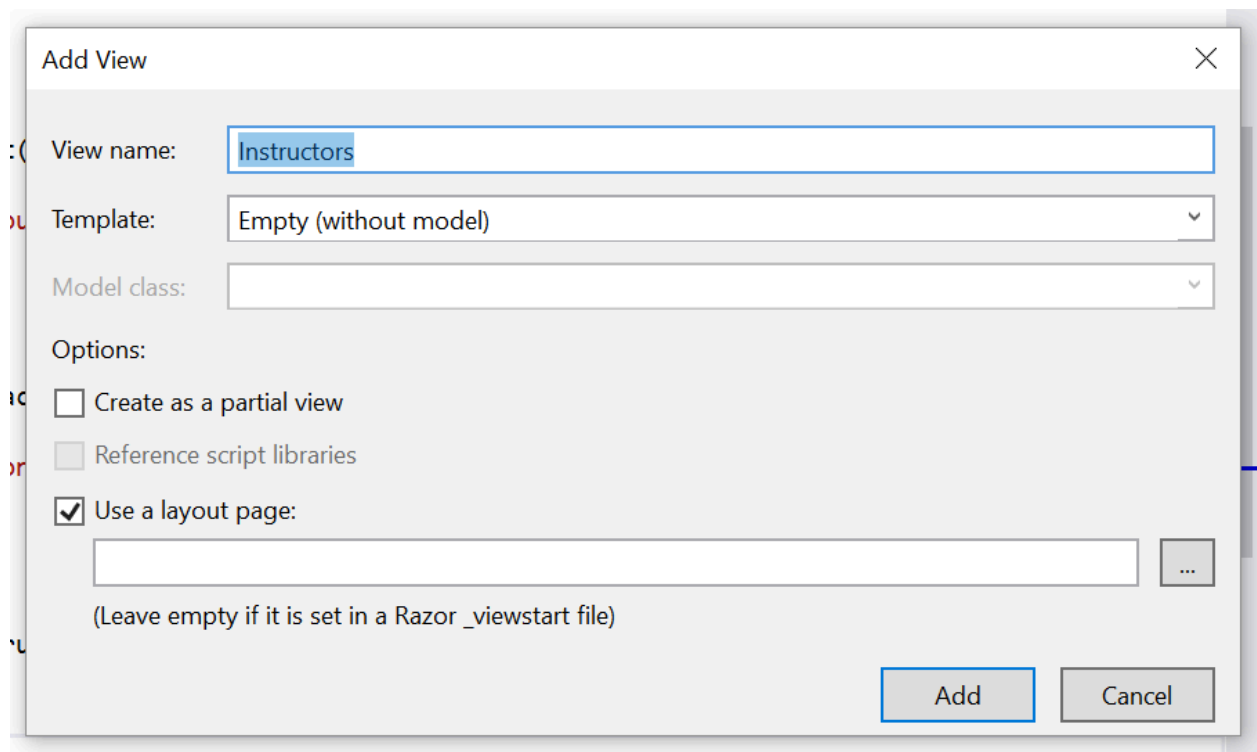
```
10 {
11     public ActionResult Index()
12     {
13         return View();
14     }
15
16     public ActionResult About()
17     {
18         ViewBag.Message = "Your application description page.";
19
20         return View();
21     }
22
23     public ActionResult Contact()
24     {
25         ViewBag.Message = "Contact Page - The Tech Academy";
26
27         return View();
28     }
29
30     public ActionResult Instructors()
31     {
32         return View();
33     }
34 }
35 }
```

Now go to `Views/Home` and right-click on the `Home` folder.

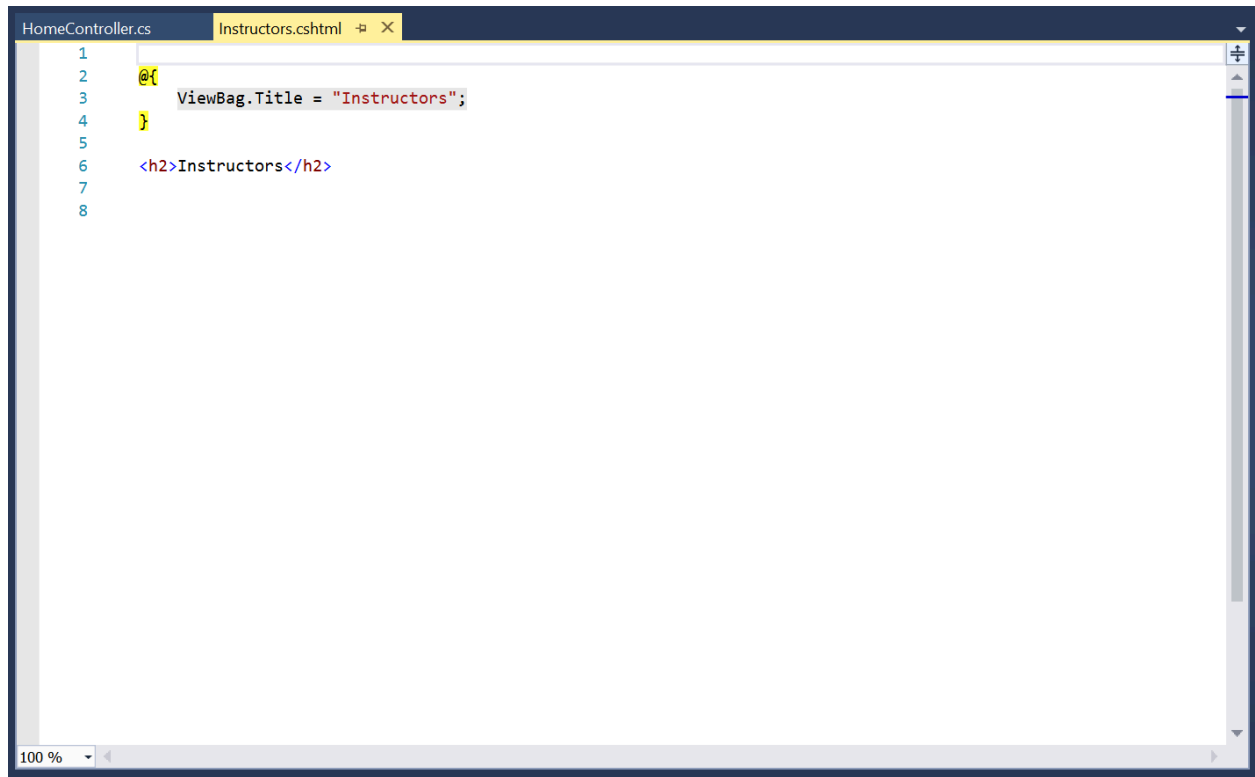
Navigate to `Add > View`.



Name the view “Instructors” and keep the default settings.



Open Instructors.cshtml if you do not already have it open from the Views/Home folder.



```
1
2 @{
3     ViewBag.Title = "Instructors";
4 }
5
6 <h2>Instructors</h2>
7
8
```

Now press F5 and start debugging.

Study these steps and perform any directed actions:

Add this C# code:



```
1 @if
2     ViewBag.Title = "Talk to us";
3 }
4 <h2>@ViewBag.Title.</h2>
5 <h3>@ViewBag.Message</h3>
6
7 <address>
8     310 SW 4th Ave Suite 200<br />
9     Portland, OR 97204<br />
10    United States<br />
11    <abbr title="Phone">P:</abbr>
12    503.206.6915
13 </address>
14
15 <address>
16     <strong>Support:</strong> <a href="mailto:Support@example.com">Support@example.com</a><br />
17     <strong>Marketing:</strong> <a href="mailto:Marketing@example.com">Marketing@example.com</a>
18 </address>
19
20 <small>
21     Contact Information last updated: @DateTime.Now
22 </small>
```

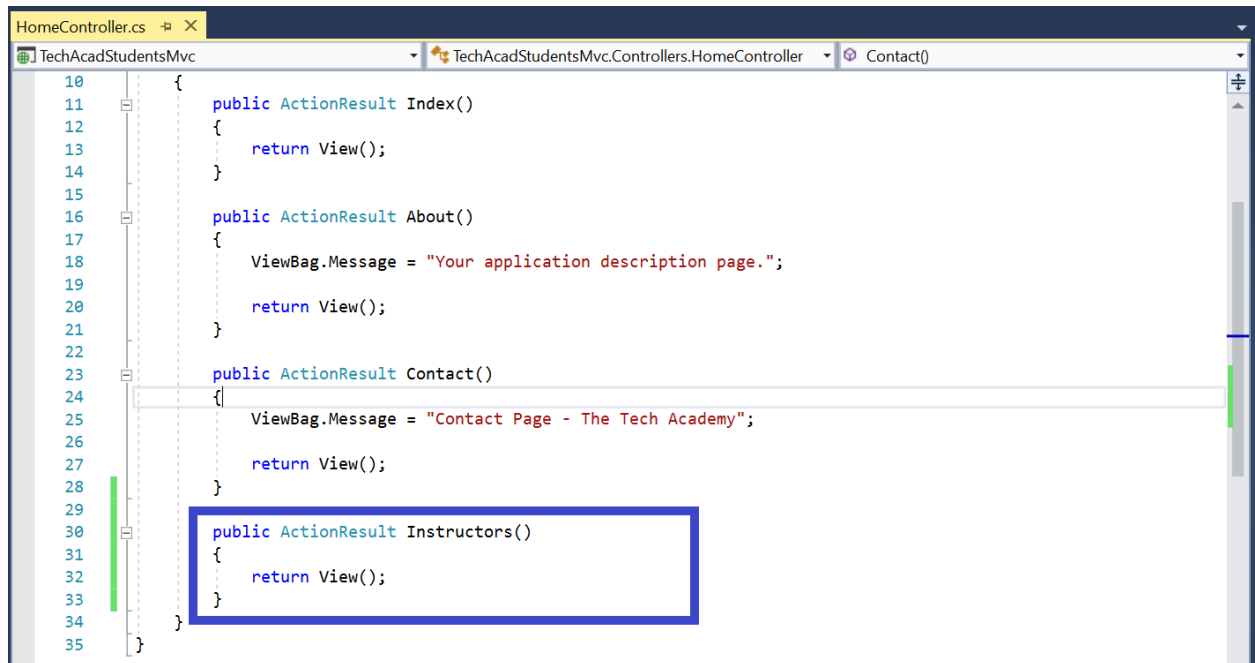
Refresh the web browser and see the result.

That's a View; we used C# in the View and we used Razor syntax.

Let's add another method to our HomeController and see what else we can do with MVC.

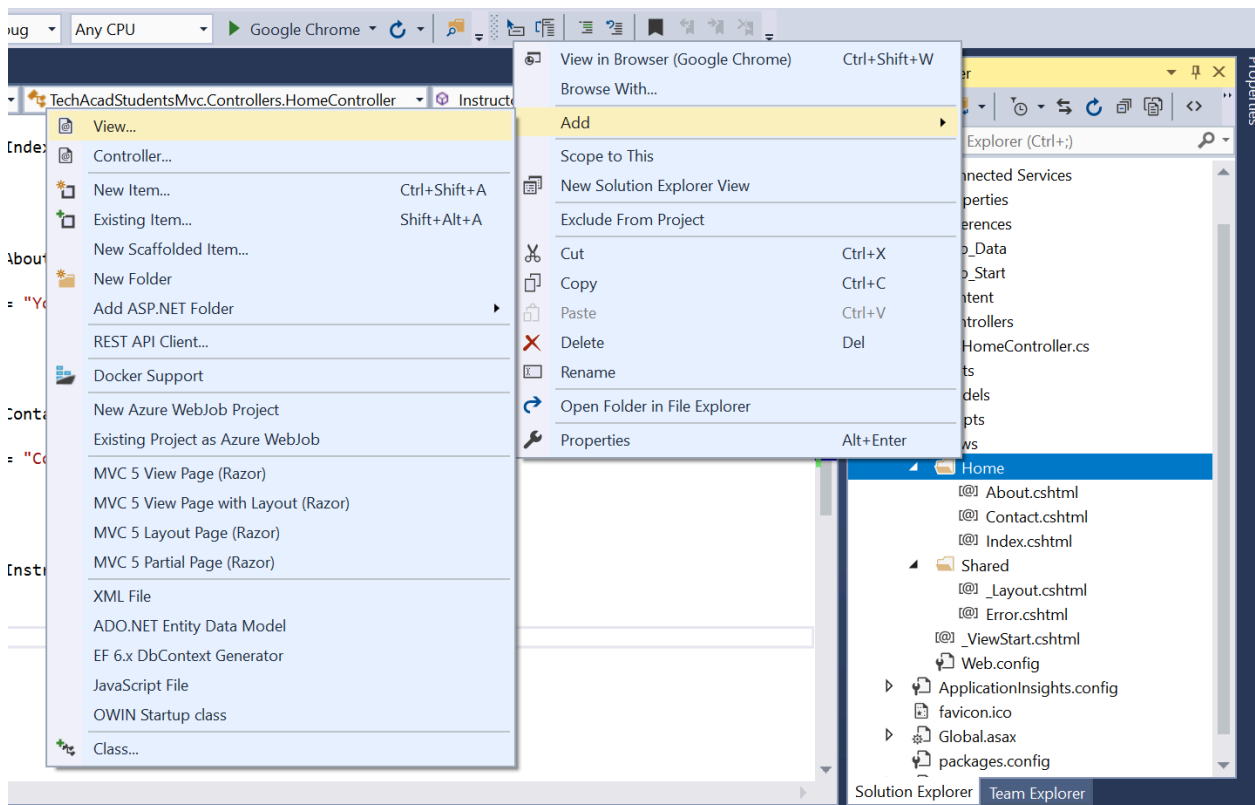
Create a method called Instructors() that takes no parameters and returns an ActionResult.

Then inside that method, just have it return View().

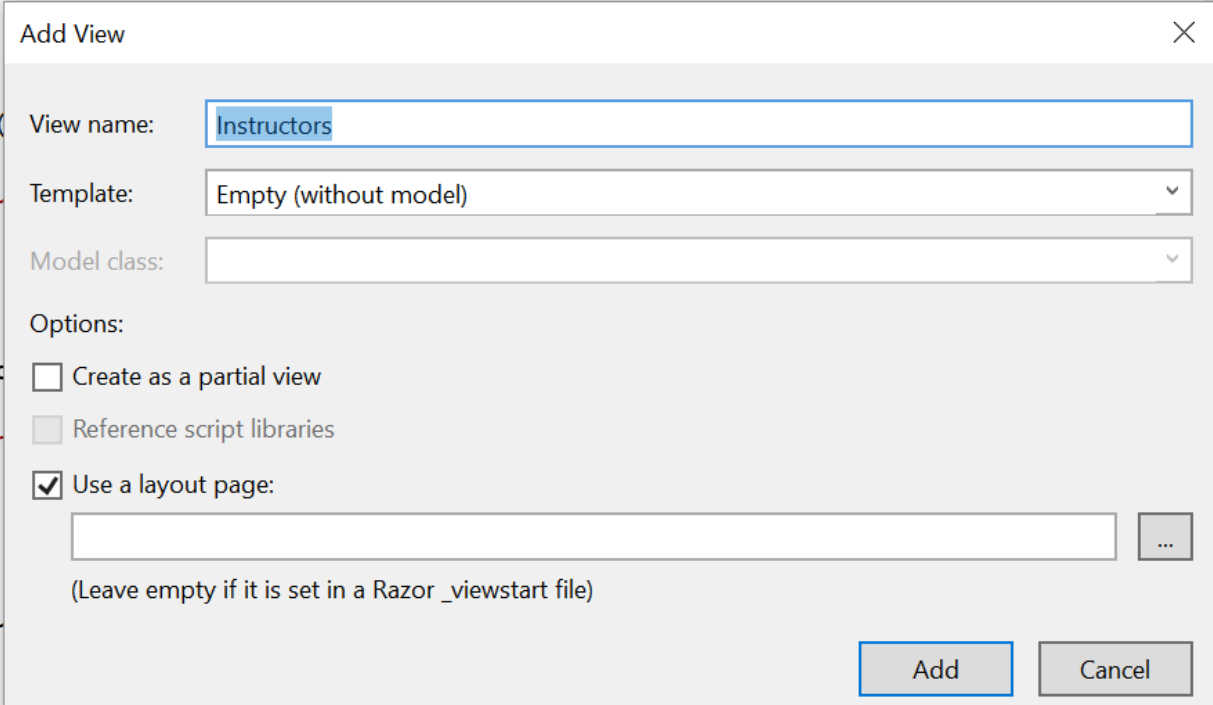


```
10 {
11     public ActionResult Index()
12     {
13         return View();
14     }
15
16     public ActionResult About()
17     {
18         ViewBag.Message = "Your application description page.";
19
20         return View();
21     }
22
23     public ActionResult Contact()
24     {
25         ViewBag.Message = "Contact Page - The Tech Academy";
26
27         return View();
28     }
29
30     public ActionResult Instructors()
31     {
32         return View();
33     }
34 }
35 }
```

Now go to Views/Home and right-click on the Home folder.
Navigate to Add > View.



Name the view “Instructors” and keep the default settings.



Add View

View name:

Template:

Model class:

Options:

☐ Create as a partial view

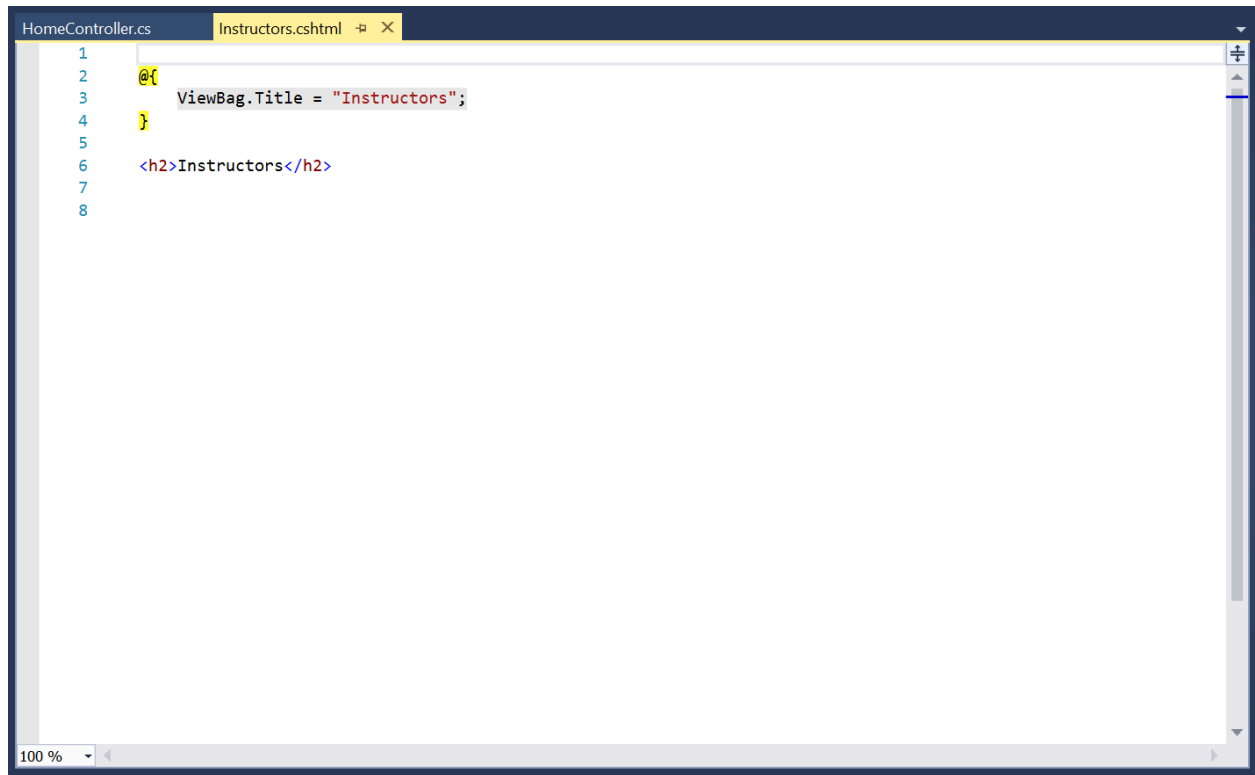
☐ Reference script libraries

☒ Use a layout page:

(Leave empty if it is set in a Razor _viewstart file)

Add Cancel

Open Instructors.cshtml if you do not already have it open from the Views/Home folder.



The screenshot shows a code editor window with two tabs: 'HomeController.cs' and 'Instructors.cshtml'. The 'Instructors.cshtml' tab is active and displays the following code:

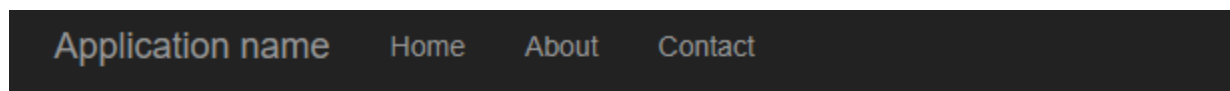
```
1
2 @f
3     ViewBag.Title = "Instructors";
4 }
5
6 <h2>Instructors</h2>
7
8
```

The code is syntax-highlighted, with '@f' in yellow, 'ViewBag.Title' in blue, and the string 'Instructors' in red. The editor has a line number margin on the left and a scrollbar on the right. The status bar at the bottom indicates '100 %' zoom.

Now press F5 and start debugging.

Study these steps and perform any directed actions:

Navigate to /Home/Instructors.



Instructors

Let's add another controller method and see how we can perform logic on the server to affect what the end user sees.

```
public ActionResult Instructor(int id)
{
    ViewBag.Id = id;
    return View();
}
```

```
public ActionResult Instructors()
{
    return View();
}
```

In order to do that, we are going to pass a parameter to the method.

In MVC, parameters are passed through the URL:

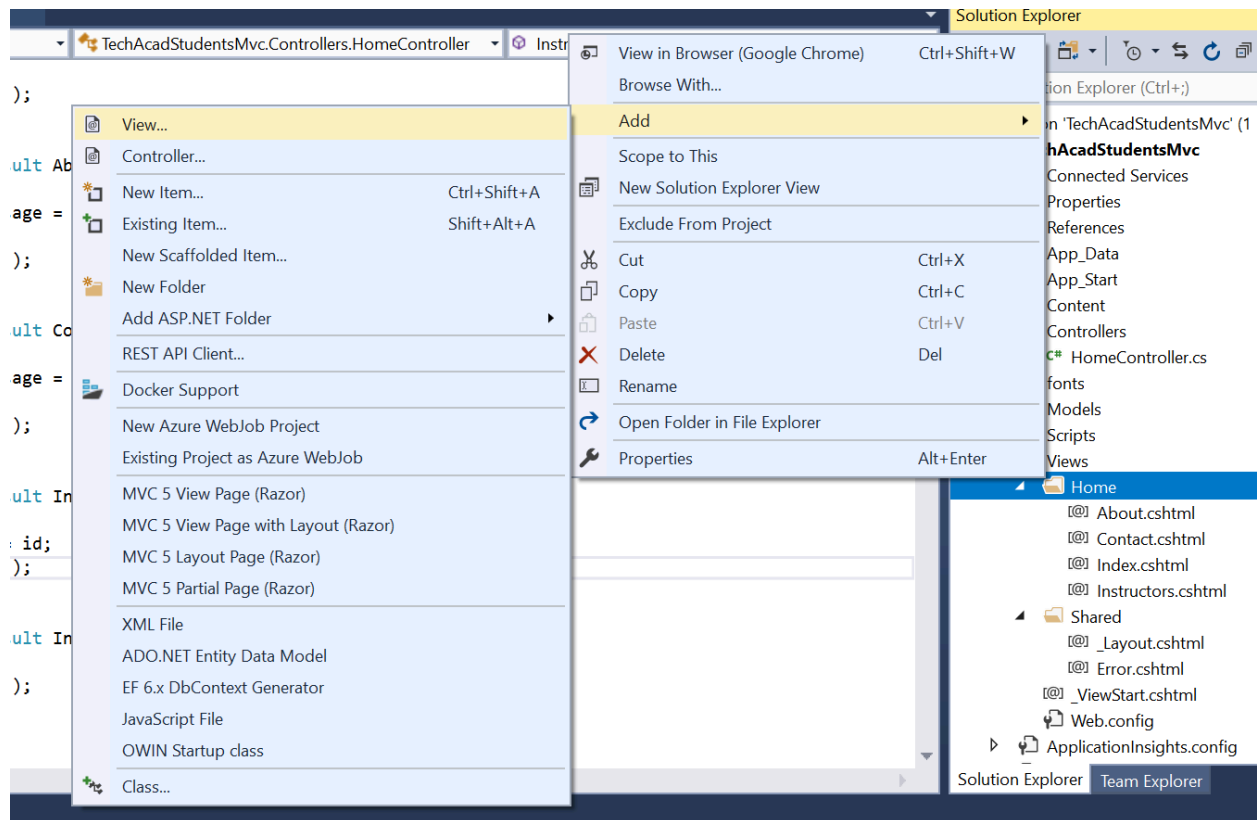
/Home/Instructors/1

or

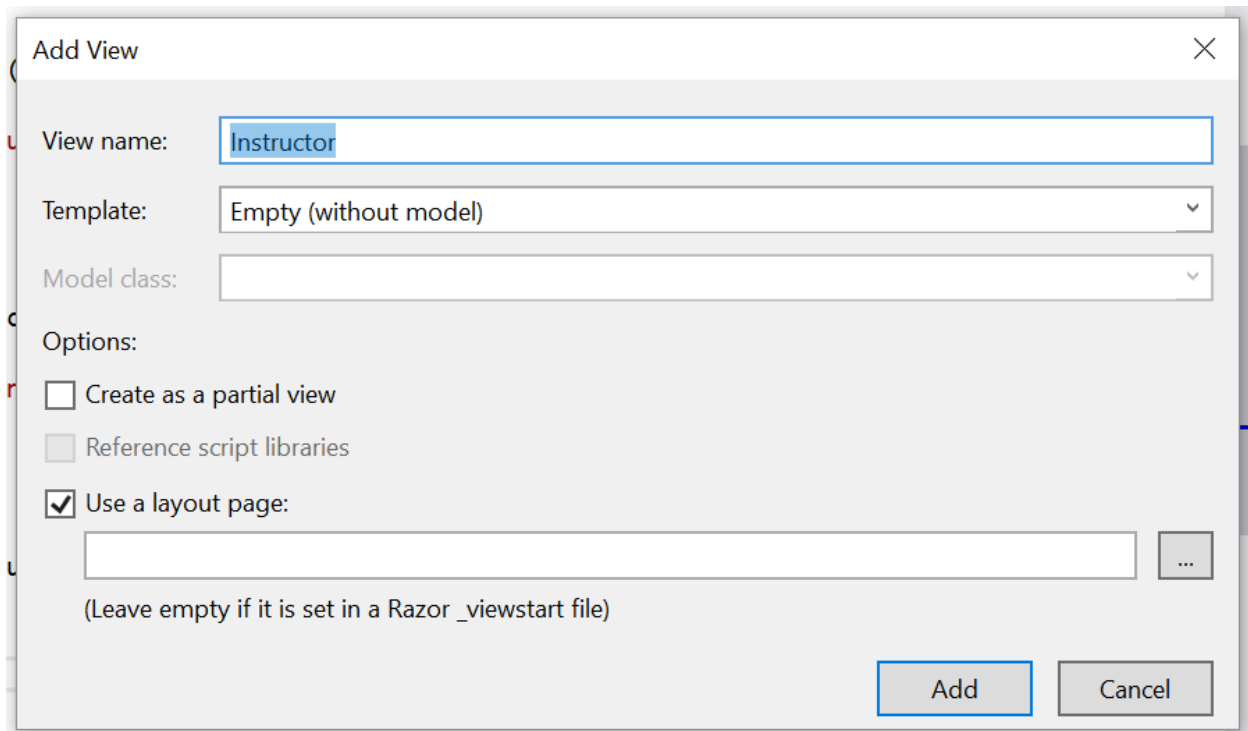
/Home/Instructors/255

where the number following the final forward-slash is typically the ID of the object you are trying to get information on.

Now let's add our Instructor.cshtml file to display the value we stored in the ViewBag. Right click the Home folder, select Add > View.



Per the MVC convention, name the file "Instructor" (the .cshtml extension will be added by Visual Studio).

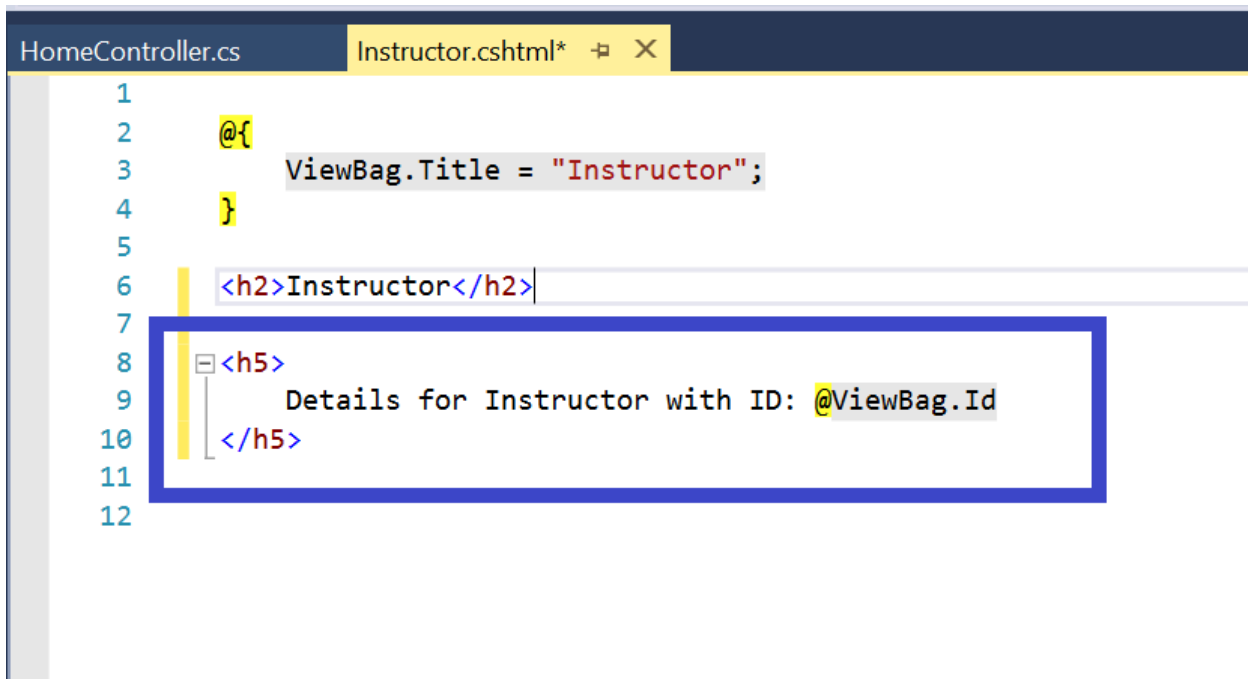


The 'Add View' dialog box is shown with the following fields and options:

- View name:** Instructor
- Template:** Empty (without model)
- Model class:** (empty)
- Options:**
 - ☐ Create as a partial view
 - ☐ Reference script libraries
 - ☒ Use a layout page:
 - (Leave empty if it is set in a Razor _viewstart file)

Buttons: Add, Cancel

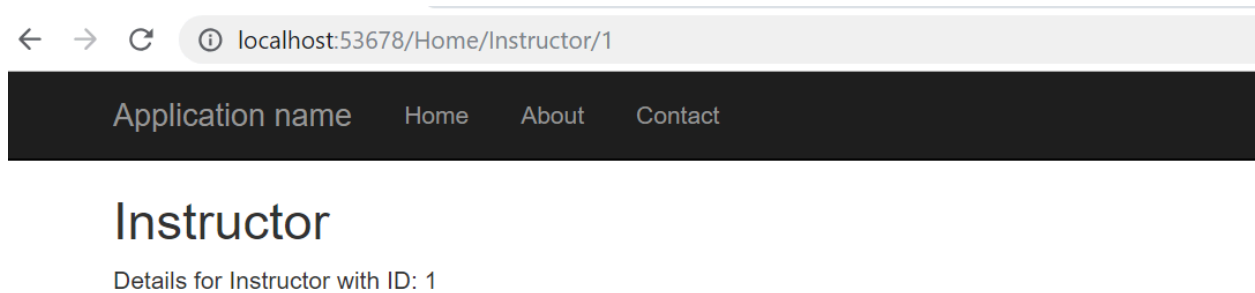
Now, if it isn't open already, open the Instructor.cshtml file and add some text, including some Razor syntax to reference the value we stored in ViewBag.



The code editor shows the following code in the Instructor.cshtml file:

```
1
2  @{
3      ViewBag.Title = "Instructor";
4  }
5
6  <h2>Instructor</h2>
7
8  <h5>
9      Details for Instructor with ID: @ViewBag.Id
10 </h5>
11
12
```

Press F5 and navigate to /home/instructor/4 and see the result.



Take a minute now to try different numbers as the “ID” for the instructor in the URL. Now try a string. What happened? (That’s a rhetorical question, but you’re free to answer it aloud to yourself).

Alright, so far we have a Controller and plenty of Views, but no Models yet.

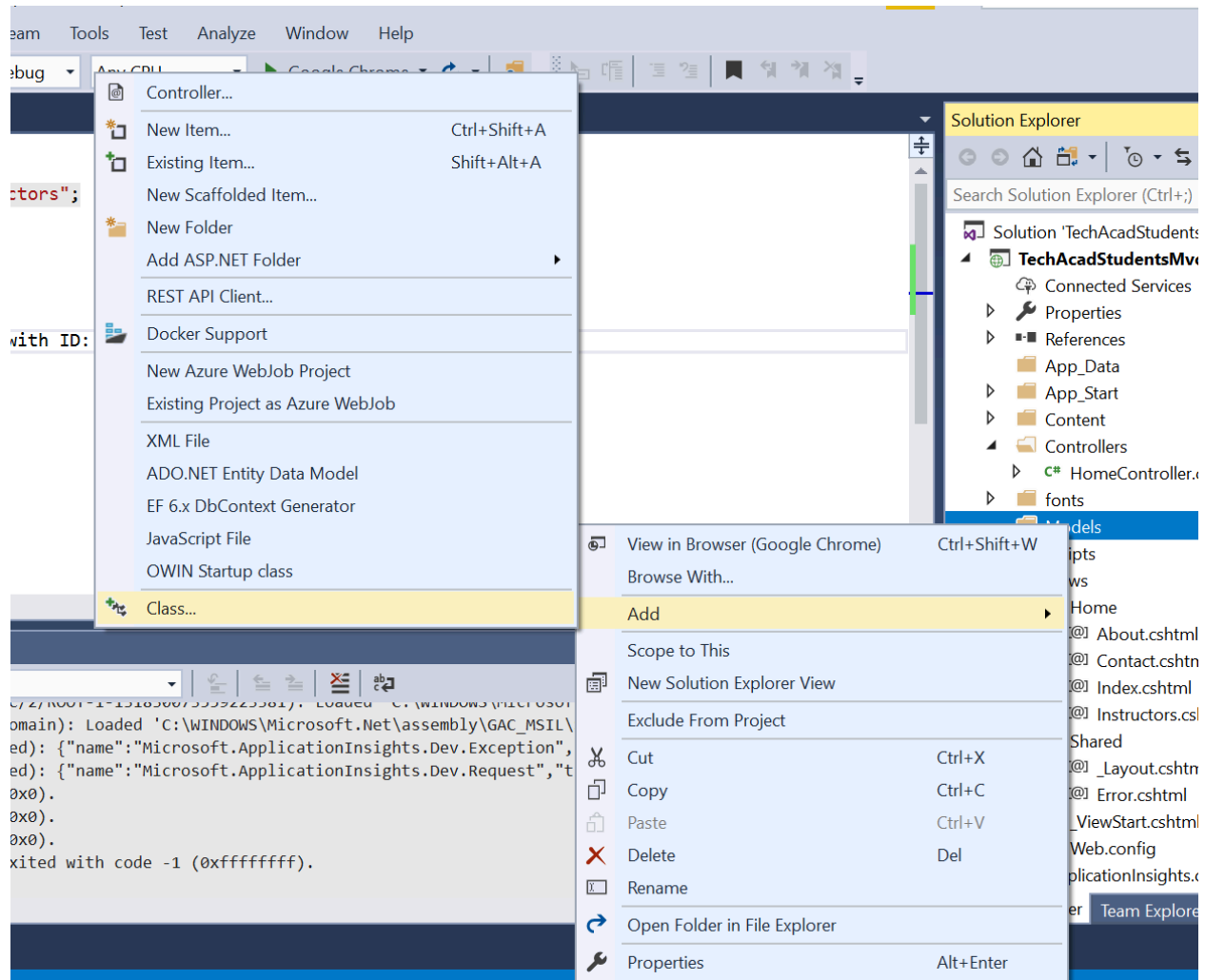
Models are the things in the domain that you are working in. Domain is a field or subject. If you were making an application for a railroad, possible models would be a train, a rail car, a passenger, a division, a route, a conductor, etc.

Models are simple C# classes (usually just properties) of these things.

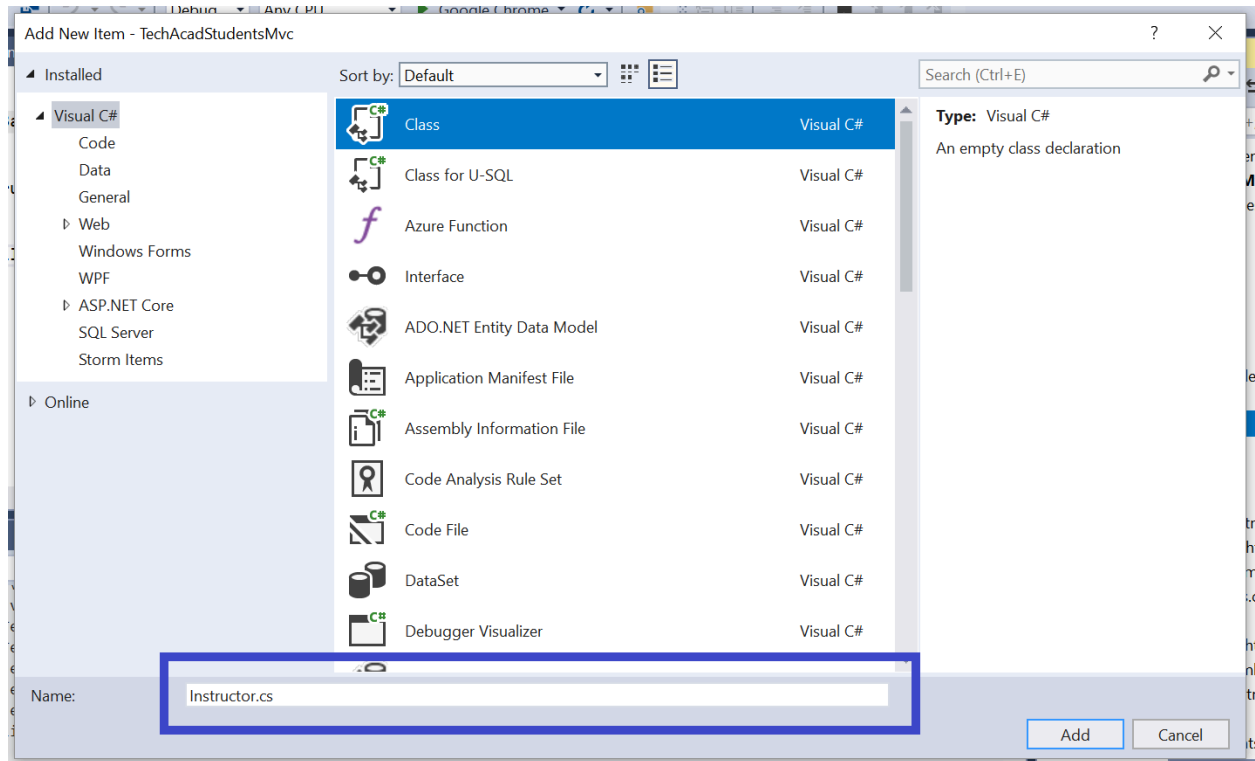
For our sample application, our domain is Coding Schools. One possible model (and very relevant to what we’re making) is an Instructor.

Study these steps and perform any directed actions:

Let’s make an Instructor model. Right-click on the Models folder and click Add > Class.

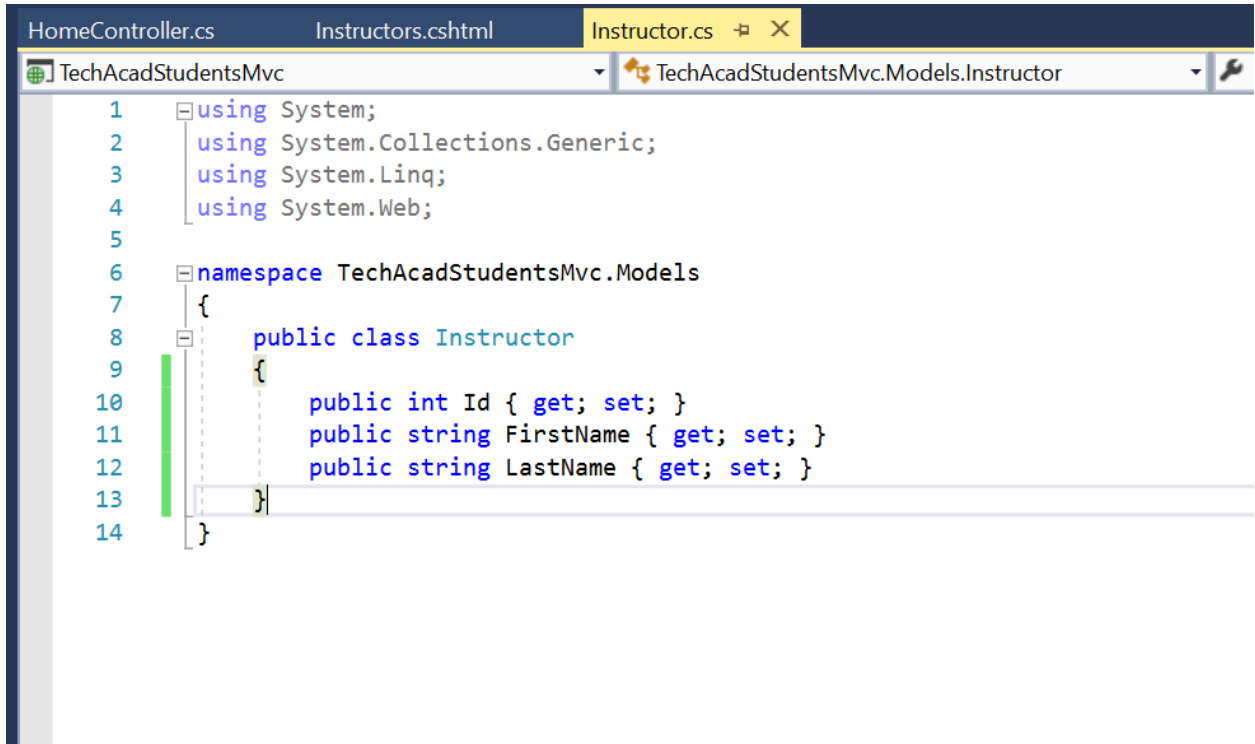


Name the Class Instructor.cs.



If it's not already open, go ahead and open the Instructor.cs file located in the Models folder.

We're going to add three properties to the Instructor class: Id, FirstName, and LastName.



```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Web;
5
6  namespace TechAcadStudentsMvc.Models
7  {
8      public class Instructor
9      {
10         public int Id { get; set; }
11         public string FirstName { get; set; }
12         public string LastName { get; set; }
13     }
14 }
```

We now have a Model that we can use in our Controllers and Views to store and display data.

Study these steps and perform any directed actions:

Let's go back to the HomeController and our Instructor method. We are going to instantiate a new Instructor object and pass it down to the View (the id parameter isn't doing much in this example, which is fine).

```

public ActionResult Instructor(int id)
{
    ViewBag.Id = id;

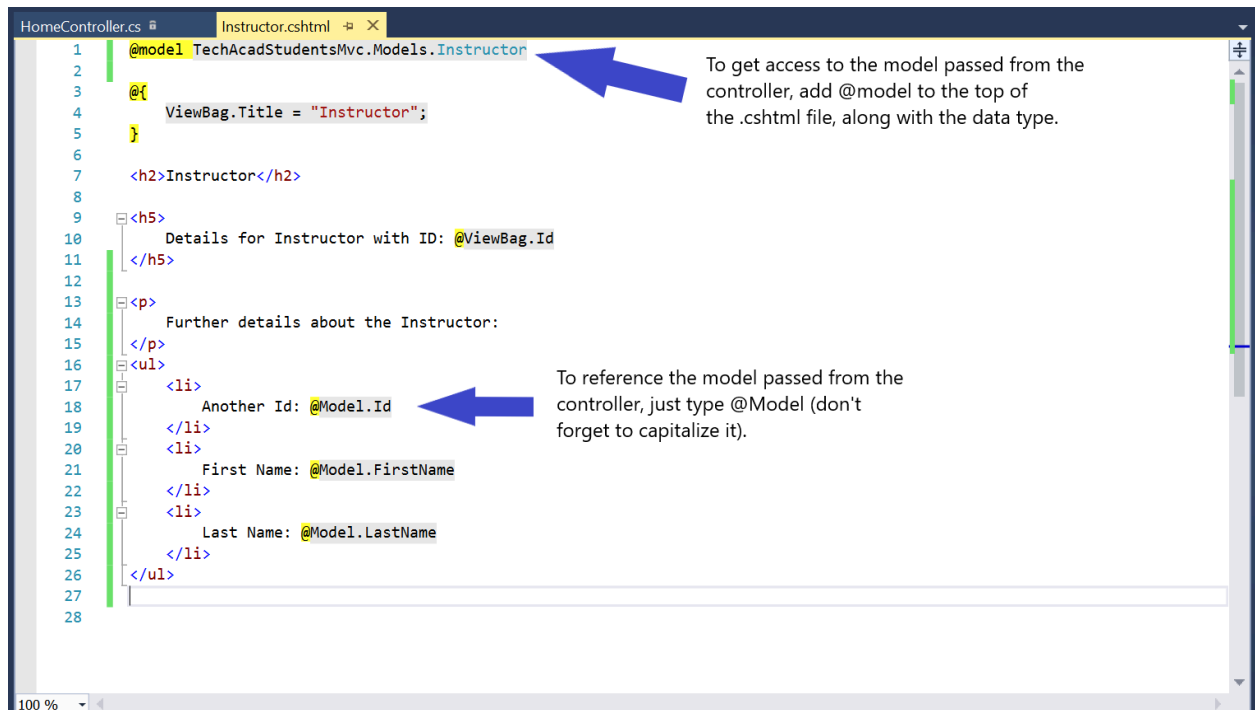
    Instructor dayTimeInstructor = new Instructor
    {
        Id = 1,
        FirstName = "Erik",
        LastName = "Gross"
    };

    return View(dayTimeInstructor);
}

public ActionResult Instructors()
{
    return View();
}

```

Now we need to edit our Instructor.cshtml file to handle this Model that we are receiving.



The screenshot shows a code editor with two files: HomeController.cs and Instructor.cshtml. The Instructor.cshtml file is open, showing the following code:

```

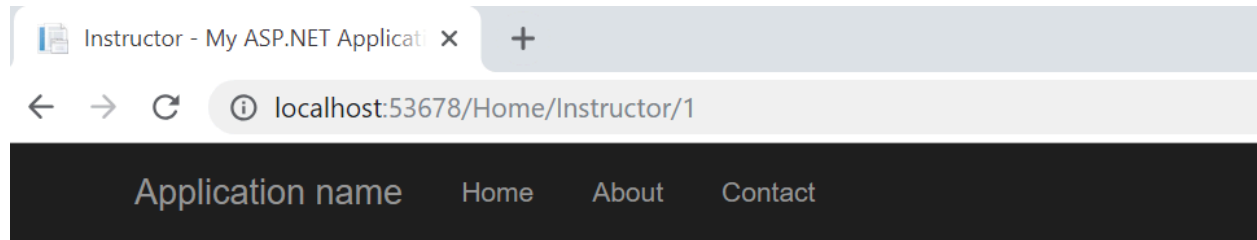
1  @model TechAcadStudentsMvc.Models.Instructor
2
3  @{
4      ViewBag.Title = "Instructor";
5  }
6
7  <h2>Instructor</h2>
8
9  <h5>
10     Details for Instructor with ID: @ViewBag.Id
11 </h5>
12
13 <p>
14     Further details about the Instructor:
15 </p>
16 <ul>
17     <li>
18         Another Id: @Model.Id
19     </li>
20     <li>
21         First Name: @Model.FirstName
22     </li>
23     <li>
24         Last Name: @Model.LastName
25     </li>
26 </ul>
27
28

```

Two blue arrows point to the annotations:

- The first arrow points to line 1, `@model TechAcadStudentsMvc.Models.Instructor`, with the text: "To get access to the model passed from the controller, add @model to the top of the .cshtml file, along with the data type."
- The second arrow points to line 18, `@Model.Id`, with the text: "To reference the model passed from the controller, just type @Model (don't forget to capitalize it)."

Now let's run it and see what it looks like. Navigate to `/Home/Instructor/1`



Instructor

Details for Instructor with ID: 1

Further details about the Instructor:

- Another Id: 1
- First Name: Erik
- Last Name: Gross

That is how you pass a Model from the Controller to the View.

Keep in mind that Model data is typically populated from a database.

The controller gives you an opportunity to call the appropriate methods to obtain that data and map it into your Model classes.

Let's go back to the HomeController and our Instructors method.

We are going to create a list of Instructors and pass them as an argument to the View method.

```

public ActionResult Instructors()
{
    List<Instructor> instructors = new List<Instructor>
    {
        new Instructor
        {
            Id = 1,
            FirstName = "Rick",
            LastName = "Ramen"
        },
        new Instructor
        {
            Id = 2,
            FirstName = "Brett",
            LastName = "Calendar"
        },
        new Instructor
        {
            Id = 3,
            FirstName = "Adam",
            LastName = "Smithsonian"
        }
    };

    return View(instructors);
}
}

```

Here we are passing our list of Instructors *to* the View.



Now let's open our Instructors.cshtml file and make a few changes:

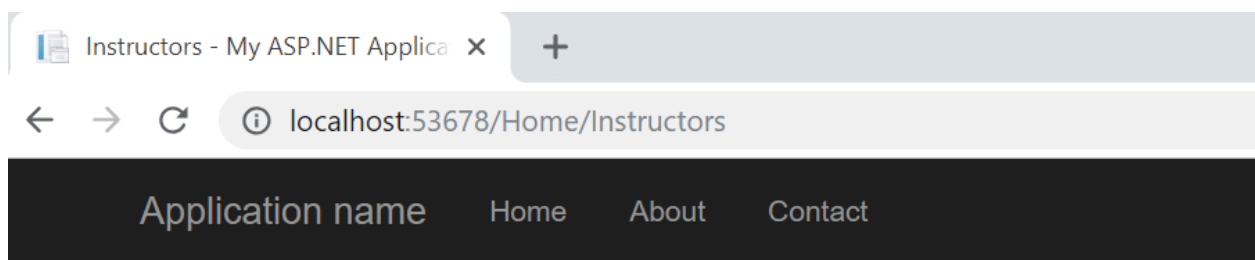
The @ sign means C# will follow. Here we are adding a using statement so we can refer to the Instructor class without prefixing it every time with TechAcadStudentsMvc.Models.

```
HomeController.cs | Instructors.cshtml
1 @using TechAcadStudentsMvc.Models
2 @model List<Instructor>
3
4 @{
5     ViewBag.Title = "Instructors";
6 }
7
8 <h2>Instructors</h2>
9
10 @{
11     foreach(Instructor instructor in Model)
12     {
13         <h4>@instructor.FirstName</h4>
14     }
15 }
16
```

Below any using statements, we define our model. "@model" means there is a model, and following it is its data type.

After you declare your model at the top with @model, you can reference it anywhere in the HTML with the word Model. As you can see in this, Razor can figure out when you are using HTML, so there is no need to close the curly brace. Just be sure to use the @ sign again to reference more C#.

Now navigate to /Home/Instructors.

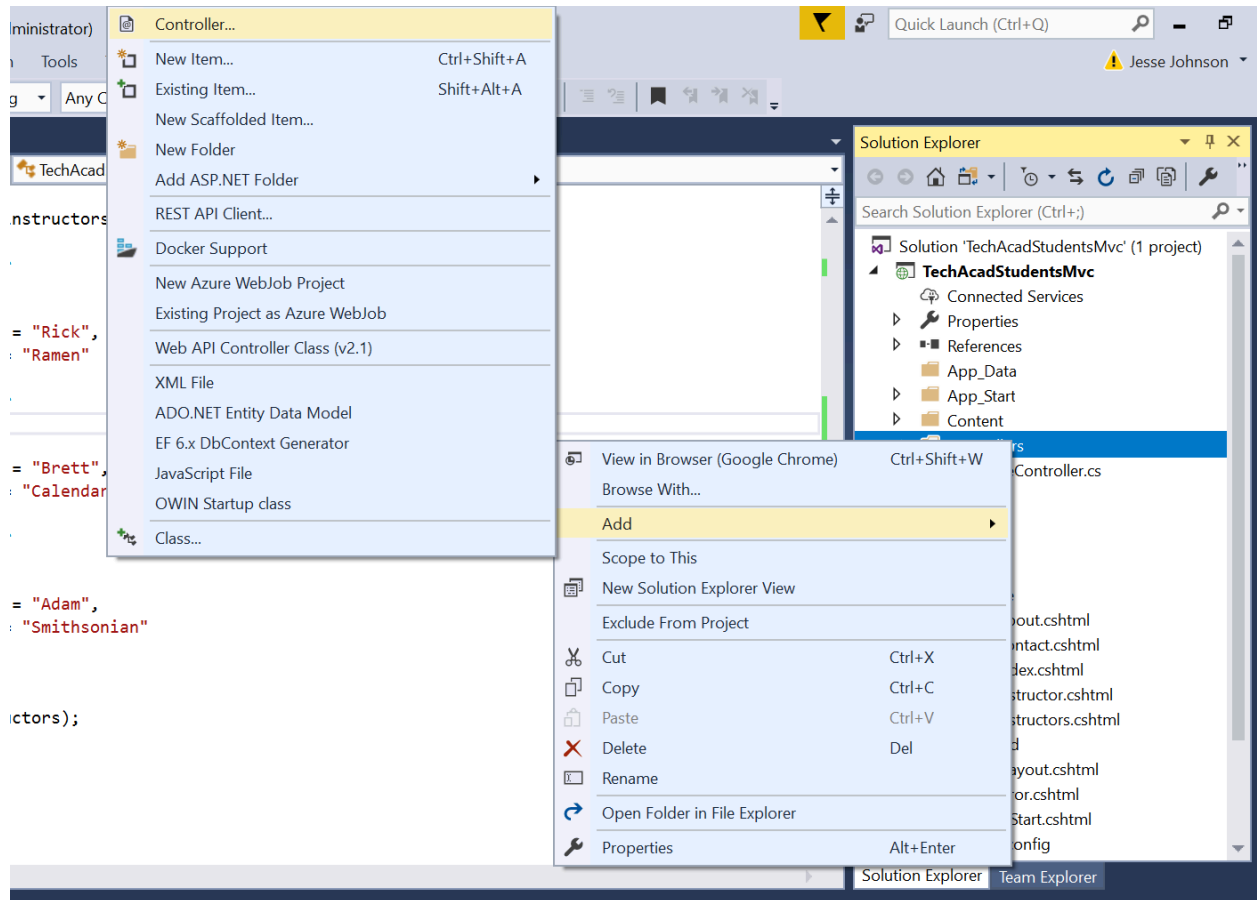


Instructors

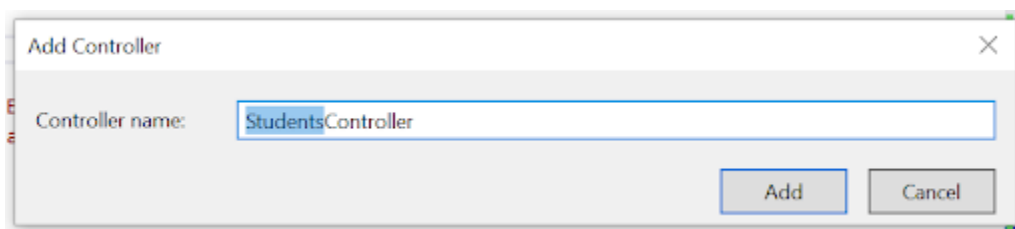
Rick
Brett
Adam

Study these steps and perform any directed actions:

To create a new Controller, right-click on the Controllers folder >
Add > Controller.



Choose the "empty" controller option, name it StudentsController and click Add.

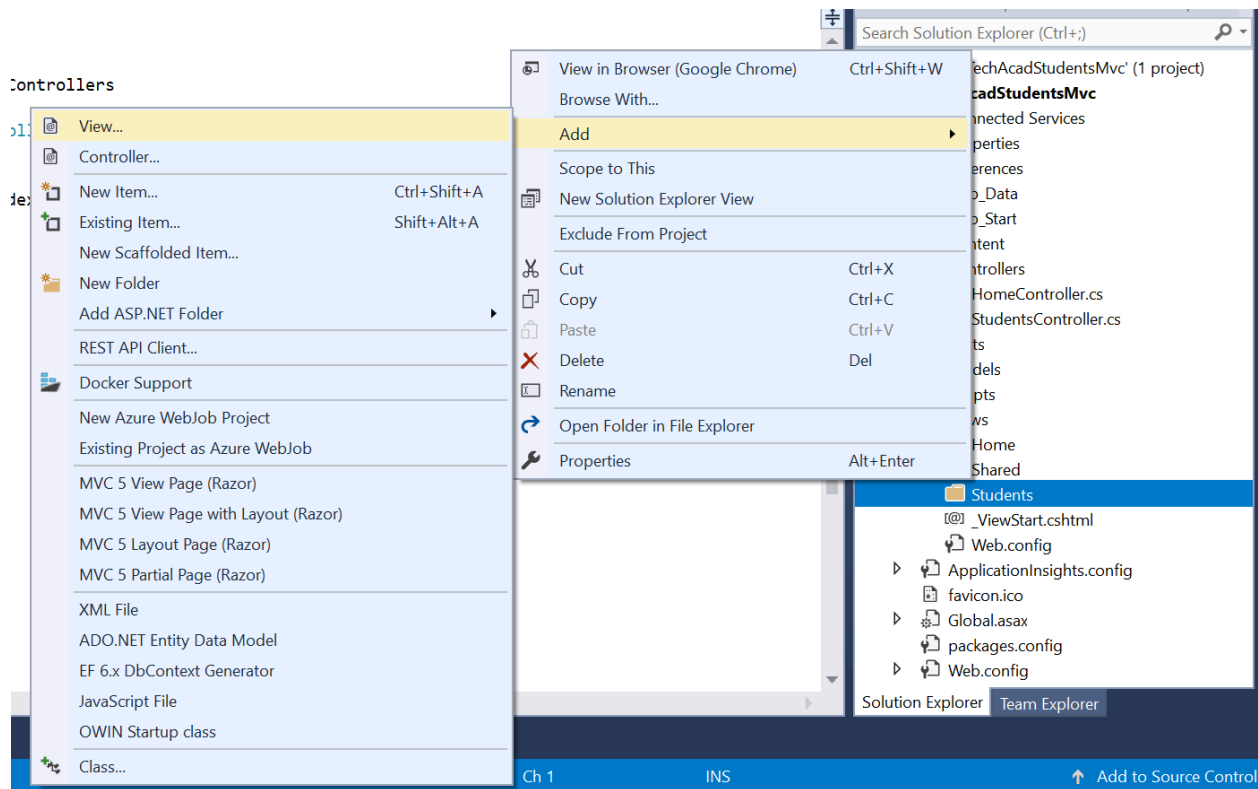


If it's not already open, go ahead and open the new StudentsController.cs file. Notice that it looks similar to the HomeController.

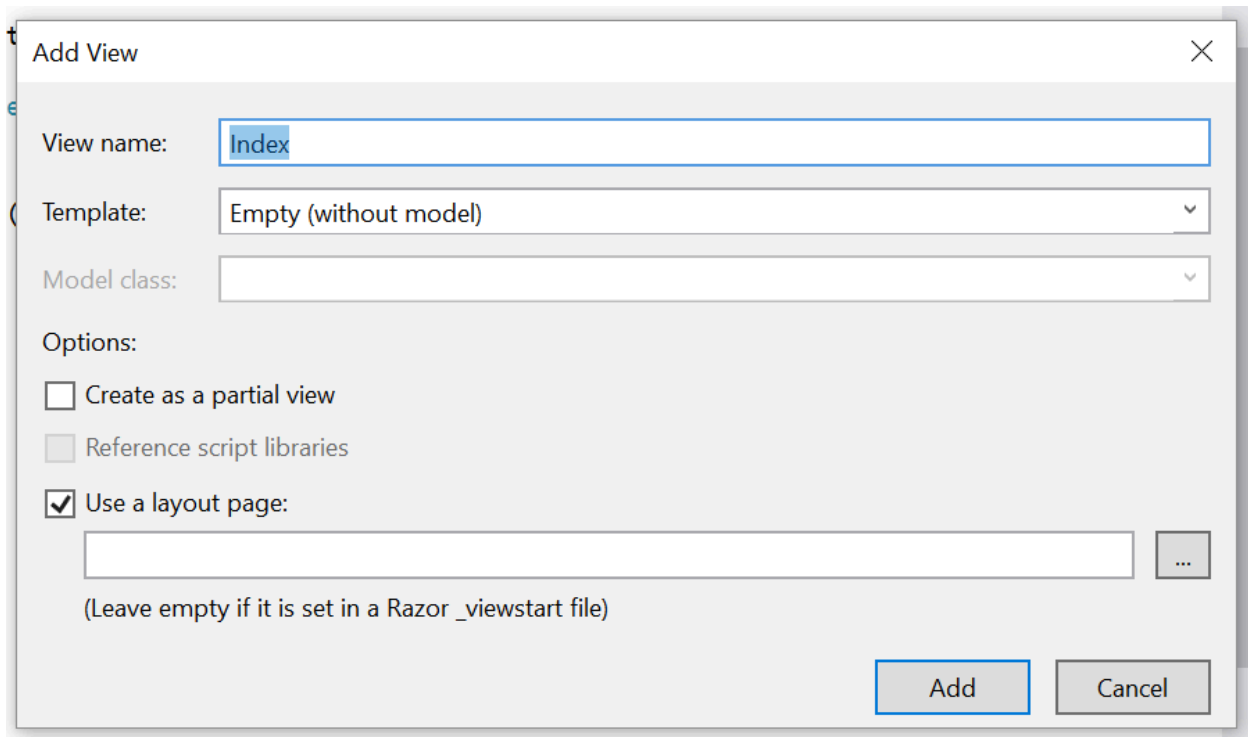
```
4  using System.Web;
5  using System.Web.Mvc;
6
7  namespace TechAcadStudentsMvc.Controllers
8  {
9      public class StudentsController : Controller
10     {
11         // GET: Students
12         public ActionResult Index()
13         {
14             return View();
15         }
16     }
17 }
```

See that it already has an Index() method created for us that returns the View() method. Also notice that it has already created the Students folder inside the Views folder. The next step for us is to create the Index.cshtml file.

Right-click on Views/Students > Add > View.



Name it Index and click Add.



The 'Add View' dialog box is shown with the following fields and options:

- View name:** Index
- Template:** Empty (without model)
- Model class:** (empty)
- Options:**
 - ☐ Create as a partial view
 - ☐ Reference script libraries
 - ☒ Use a layout page:
 - (Leave empty if it is set in a Razor _viewstart file)

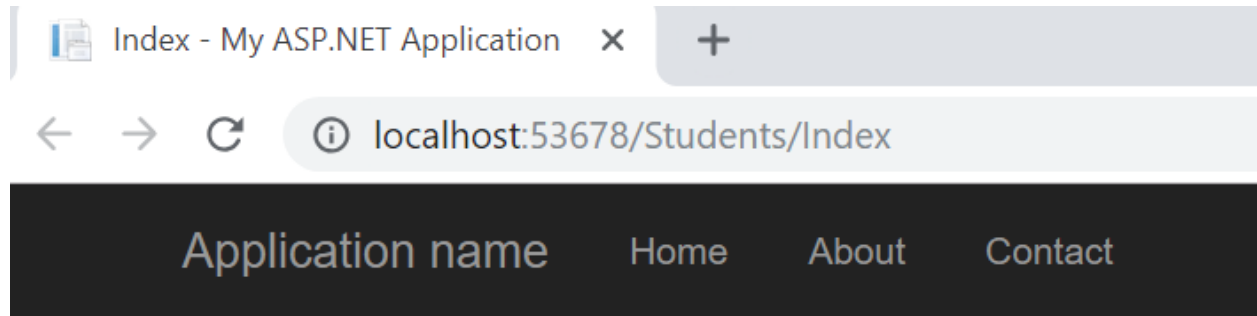
Buttons: Add, Cancel

If it's not already open, go ahead and open the Views/Students/Index.cshtml file.



```
1
2  @{
3      ViewBag.Title = "Index";
4  }
5
6  <h2>Index</h2>
```

Go ahead and run the application and navigate to /Students/Index.



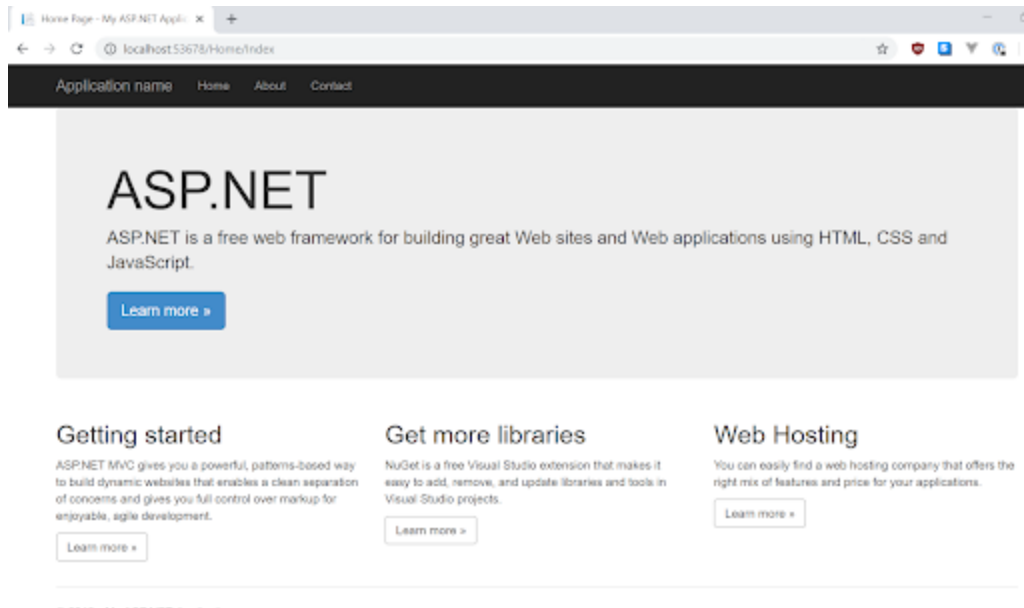
Index

What's happening here?

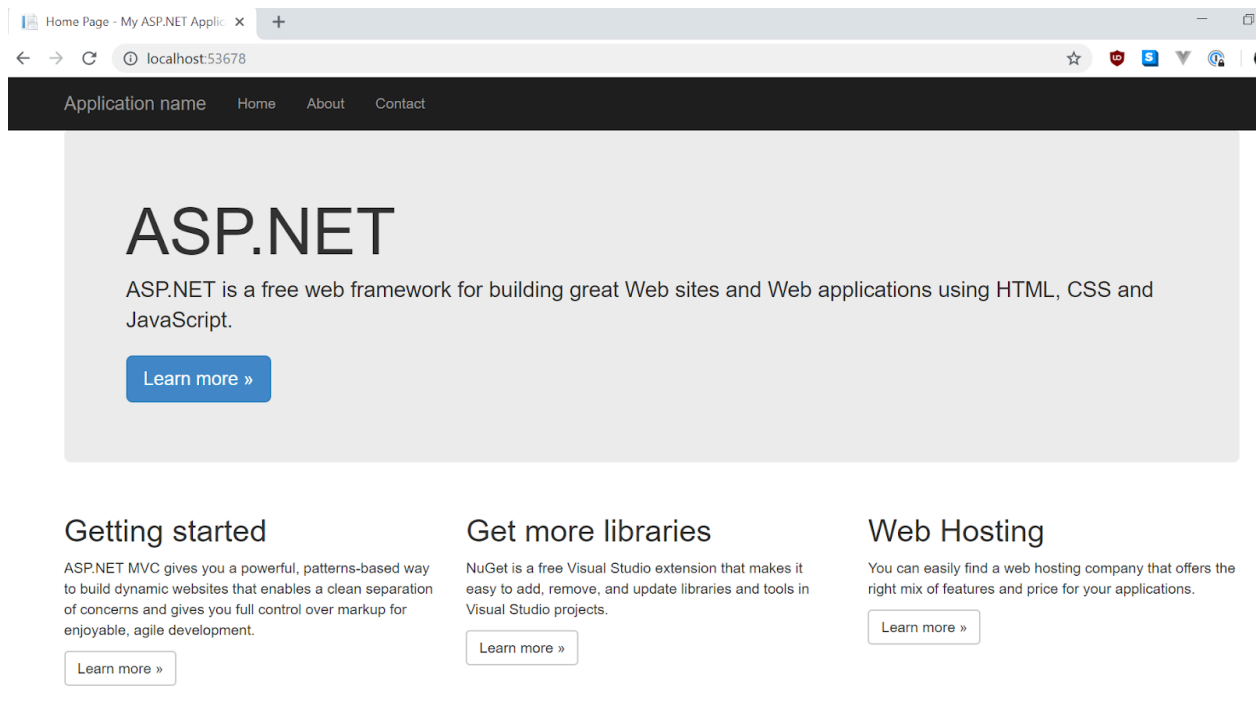
MVC has defined default routes. Every Controller has a default route of Index, meaning if no method is specified after the controller name in the URL, it will default to Index.

Furthermore, the root URL of the entire site has a default route of /Home/Index.

To verify this, navigate to /Home/Index.



Now navigate to just the root of the application (no slashes after the “localhost:12345”).



It's the same exact view.

Try navigating to just /Home.

Congratulations! You created a basic MVC application!

NuGet is the Microsoft supported mechanism for sharing libraries and packages of code for .NET Framework and .NET Core. NuGet defines how packages for .NET are created, hosted, and shared.

NOTE: In the video, you are instructed to upgrade bootstrap to the latest stable version. However, currently, the latest stable version of bootstrap will throw an error and break the program. To prevent this from occurring, please upgrade your version of bootstrap to the one that is used in the video, version 4.1.1.

Jquery and bootstrap project

Form control - Newsletter Project using MVC and SQL

Advanced Server utilization in projects

Connection Strings*

Business Object*

Refactoring*

Unit Testing*

QA Software*

What is the level of skill required to learn and use the software?

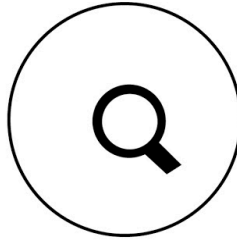
How long does it take to begin using the software?

What is the user's attitude toward the software?

How does the user respond to the software under realistic conditions?



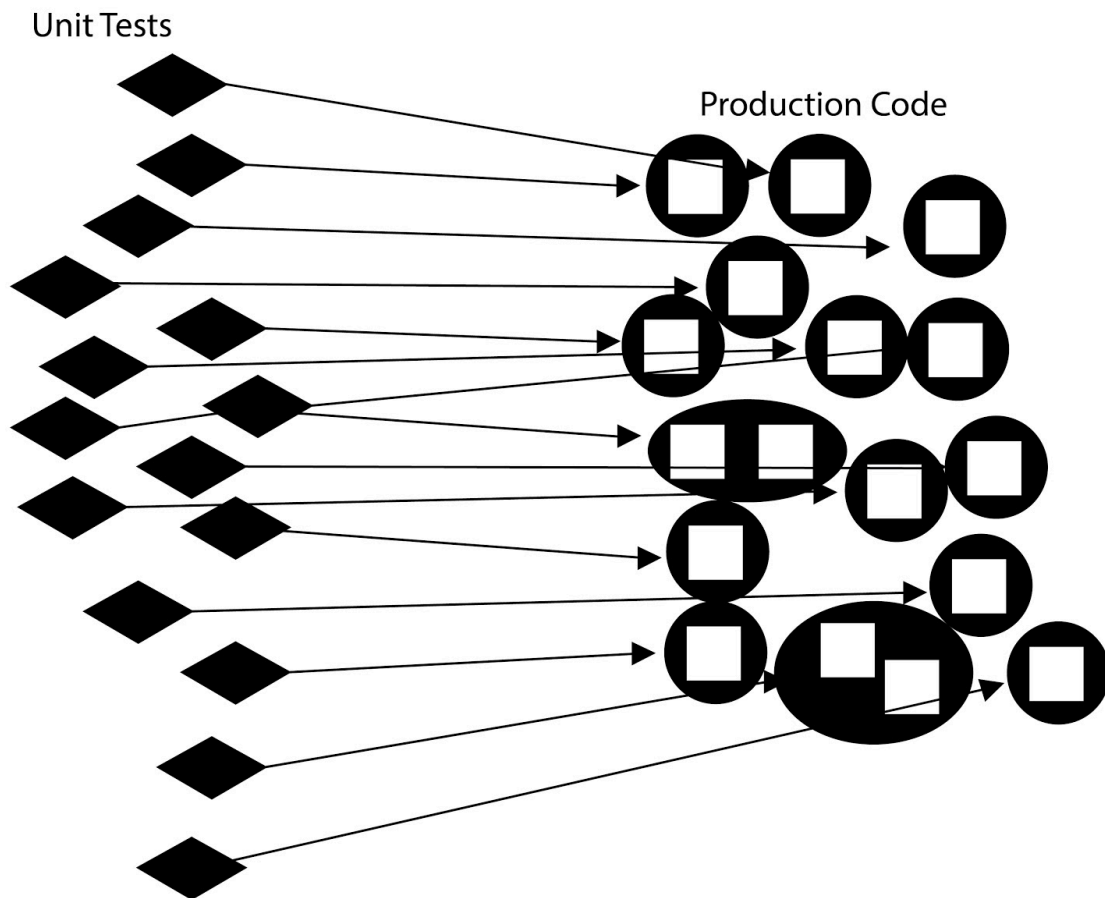
Uncover Problems
in the design



Discover Opportunities
to improve the design



Learn About Users
behavior and preferences



Portfolio Idea - Create a QA Checklist and familiarize self with QA software utilization

MAPPING

In general, mapping is the creation of a system that connects one thing to another – essentially, it answers the question “how do I get from here to there.”

Object Relational Mapping is abbreviated “O/RM.” It is a program that maps between objects in a program and tables in a database.

In the database, you will find tables of data along with descriptions of how the data relates to other data in the database.

In the computer program, you will find objects - custom data types created by the computer programmer. An example might be a “Student” object.

Often, the tables in a database correspond to types of objects in the program. Each row in a particular table might correspond to a particular object of a certain type. Using our example, there might be a “Student” table.

As the program operates, it may change the state of a particular object. In order to keep a record of that object’s new state, you might want to make a change to the row in the database that represents that particular object. Continuing our example: If the computer program were to recalculate a student’s grade point average, you might want to store that new GPA permanently in the database.

Writing code in your program that does this synchronization, or mapping is time-consuming. It is a very common thing that's needed in many situations.

This task is made easier through the use of an Object/Relational Mapper program. It can keep track of the mapping between various object types in a program, and the corresponding data in a database, and ensure they are kept in sync.

Nuget and Entity Framework

(getting and updating information with Framework)

DB First vs Code First

Application Project

Open Visual Studio and click File > New > Project.

Select Visual C# > ASP.NET Web Application (.NET Framework).

Name the project "CarInsurance", then click Create.

Select the MVC template, then click Create.

In the Solution Explorer, right click the App_Data folder and select Add > New Item.

Select SQL Server Database and name your database “Insurance.mdf,” then click Add.

In the Solution Explorer, double click Insurance.mdf.

In the Server Explorer, under Insurance.mdf, right-click the Tables folder and select Add New Table.

In the T-SQL screen type the following SQL query:

```
CREATE TABLE [dbo].[Insurees]
(
    [Id] INT NOT NULL PRIMARY KEY IDENTITY (0,1),
    [FirstName] NVARCHAR(50) NOT NULL,
    [LastName] NVARCHAR(50) NOT NULL,
    [EmailAddress] NVARCHAR(50) NOT NULL,
    [DateOfBirth] DATETIME NOT NULL,
    [CarYear] INT NOT NULL,
    [CarMake] NVARCHAR(50) NOT NULL,
    [CarModel] NVARCHAR(50) NOT NULL,
    [DUI] BIT NOT NULL,
    [SpeedingTickets] INT NOT NULL,
    [CoverageType] BIT NOT NULL,
    [Quote] MONEY NOT NULL
)
```

Click Update, then click Update Database.

In the Solution Explorer, right click the Models folder and select Add > New Item.

Select ADO.NET Entity Data Model, name your data model “InsuranceEntities,” then click Add.

Select EF Designer from the database, then click Next.

Ensure that your data connection is to Insurance.mdf and that you're saving connection settings in Web.Config as “InsuranceEntities,” then click Next.

If prompted, select Entity Framework 6.x, then click Next.

Click Tables, then click Finish.

Build the project (CTRL + SHIFT + B).

In the Solution Explorer, right click the Controllers folder and select Add > Controller.

Select "MVC 5 Controller with views, using Entity Framework", then click Add.

From the Model class dropdown list, select Insuree (CarInsurance.Models).

From the Data context class dropdown list, select InsuranceEntities (CarInsurance.Models).

Change the controller name from "InsureesController" to "InsureeController," then click Add.

You now have a multi-page website with full CRUD functionality and the five standard MVC Views, each of which comes with the following built-in functionality:

Index: The Index View of a given Controller presents a page consisting of a list of all objects of the Model from which that Controller was scaffolded.

Create: The Create View of a given Controller presents a page with which the user can instantiate an object of the Model from which that Controller was scaffolded.

Edit: The Edit View of a given Controller corresponds to one object of the Model from which that Controller was scaffolded; it allows the user to change the values of that object's properties.

Details: The Details View of a given Controller corresponds to one object of the Model from which that Controller was scaffolded; it allows the user to view the values of that object's properties.

Delete: the Delete View of a given Controller corresponds to one object of the Model from which that Controller was scaffolded; it allows the user to delete that object.

Build the project and press F5 to run the application in the browser.

In the browser bar, change the URL to <http://localhost:12345/Insuree/Index> (your number won't be 12345; use whatever number is pre-populated in your browser bar).

If you have closed down the browser with your project, rebuild it now.

Click Create New (this takes us to the Create View).

Enter the following values (note that all values must be provided since none are NULLABLE. Values for non-string data types must follow a specific format, or the application will yield a model error to the user):

FirstName: Joe

LastName: Black

EmailAddress: joeblack@gmail.com

DateOfBirth: 09/28/1991

CarYear: 2013

CarMake: Ford

CarModel: C-Max SE

DUI: (don't click this)

Speeding Tickets: 0

Coverage Type: (don't click this)

Quote: 131.98

Click Create. We're back at the Index View. Now there's one record in it! In the Server Explorer, if you right-click the Insurees table and select Show Table Data, you will see that the entry for Joe Black has been persisted to the database. In the Index View entry for Joe Black, click Edit.

In the Edit View, change the value in the DateOfBirth field from 9/28/1991 12:00:00 AM to 9/28/1991 11:00:00 PM, then click Save. We've returned back to the Index View and our record for Joe Black now reflects the new value in its DateOfBirth field. In the Server Explorer, if you right click the Insurees table and select Show Table Data, you will see that the entry for Joe Black has been modified in the database as well.

In the entry for Joe Black, click Details. Take a look around the Details View, then click Back to List.

In the Index View entry for Joe Black, click Delete. Let's say he's switching to Geico and is no longer a client of CarInsurance.

We are now in the Delete view. Click Delete here to confirm the deletion of the object. In the Server Explorer, if you right-click the Insurees table and select Show Table Data, you will see that the entry for Joe Black has been removed from the database.

Note that the Index View is empty once more.

In an ASP.NET MVC project like this one, your Models, whether hard-coded or mapped from a database using EF Designer from Database, are just classes. Usually these

classes correspond to objects that a user or administrator of your application can create, edit, view the details of, and delete.

Your Controllers are highly-specialized classes, which are responsible for the functional logic of your program. In your `InsureeController` will go most of the extra functions you'll write to further refine this project (e.g., the function that will automatically calculate a quote from the user's other inputs rather than allowing the user to input their own quote).

Your Views are just web-pages built for a specific purpose. The 5 standard MVC views (Index, Create, Edit, Details, Delete) often provide the bulk of the functionality that a simple database-driven website will need, but you can also build your own Views (and Models and Controllers) for additional functionality in more complex applications.

From this point, by editing the code in your `InsureeController` and `Insuree Views` (and maybe, if you're a perfectionist, by adding an `InsureeMetadata` class to your Models, and even a CSS file for styling your application), you should be able to meet the parameters of this assignment.

In the InsureeController, add logic to calculate a quote based on these guidelines:

Start with a base of \$50 / month.

If the user is 18 or under, add \$100 to the monthly total.

If the user is from 19 to 25, add \$50 to the monthly total.

If the user is 26 or older, add \$25 to the monthly total.

Double check your code to ensure all ages are covered.

If the car's year is before 2000, add \$25 to the monthly total.

If the car's year is after 2015, add \$25 to the monthly total.

If the car's Make is a Porsche, add \$25 to the price.

If the car's Make is a Porsche *and* its model is a 911 Carrera, add an *additional* \$25 to the price. (Meaning, this specific car will add a total of \$50 to the price.)

Add \$10 to the monthly total for every speeding ticket the user has.

If the user has ever had a DUI, add 25% to the total.

If it's full coverage, add 50% to the total.

Modify the Create View so that the user does not see the "Quote" input field.

Add an Admin View for a site administrator to the Insuree Views. This page must show all quotes issued, along with the user's first name, last name, and email address.

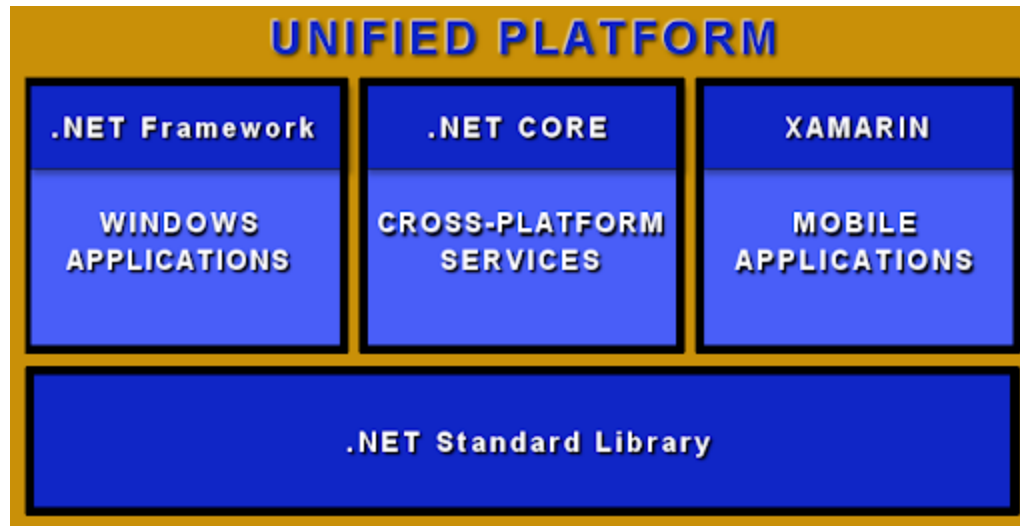
Upload your code to GitHub.

***Xamarin* is software from Microsoft that allows for cross-platform development.**

Not long after Xamarin was developed, Microsoft took interest in all the possibilities that Xamarin could offer in their quest to make their cross-platform framework.

With Xamarin's open-source version of the .NET Framework and their own original .NET Framework, Microsoft began to fill in the puzzle to create a unified platform.

Microsoft elected to create an all new and open-source version of their framework to support cross-platform services naming it “.NET CORE.” Then they expanded their original “.NET Standard Library” into the unified platform. This unified platform would encompass their original .NET Framework and extend beyond the Windows OS to also include Linux and the Mac operating systems.



With .NET Core, you will have three deployment options:

Framework-dependent:

With this option, your app contains only its own code and any third-party dependencies that are outside of the .NET Core libraries. Your framework-dependant application will require that the target computer has already installed the appropriate version of the .NET Core Library. With this option, you will have a .dll file that can be launched during the .NET Core runtime.

Self-contained:

This particular option will not rely on the system nor any of its installed

files. Instead, a self-contained version for deployment is a package containing all files necessary to run the application including both the .NET Core libraries and the .NET Core runtime.

Framework-dependent executables:

This option for deployment will generate an executable file that will run on *only* the specific operating system platform. These framework-dependent deployments still rely on the presence of a shared system-wide version of .NET Core to run. This option will still require that the target has already installed the appropriate version of the .NET Core Library.

Razor Pages is a page-focused (front-end-centric) framework for creating dynamic, data-driven websites without the complexity of MVC. The framework is built on top of ASP.NET Core MVC. A Razor Page is very similar to the View component of MVC, the main difference is that the Model and Controller code are included in the Razor Page itself. The Razor Pages framework is lightweight and allows the developer to have full control over rendered HTML.

The file extension for a Razor Page is .cshtml (a combination of .cs and .html). Razor Pages also have their own syntax. Some of the main rules of the Razor syntax are:

- Code blocks are enclosed in @{ ... }

- Inline expressions such as variables and functions are denoted with @

- Variables are declared with the var keyword

- C# code is case sensitive

Below is an example of a Razor Page:

```
@{ var age = 18;}
<html>
  <body>
    @if (age > 18)
    {
      <p>You need to be 18 or older to create an account.</p>
    }
  </body>
</html>
```

The Razor Syntax (@{ ... } and @) is used to denote the C# code. The Razor Page will take the complex task written in C# and render its output to the browser in an HTML paragraph element.

Before you can use ASP.NET Core, you will need to ensure that your computer is appropriately set up as per [Microsoft's documentation](#) on the subject.

A *container* is an object that can be used to hold or transport something. In computing, a container is used to hold and deliver software. The container allows the application to run quickly and reliably from one computing environment to another.

Docker creates containers by encapsulating the program with all of the necessary libraries and configuration files into a self-contained package. Software developers can then access these containers and have a fully running computer program on any operating system.

Review and study Docker

self-teach and debug are *vital* to being a successful developer.

The purpose of this challenge is not to give you further experience in writing C# or Razor, it's to teach you how to spin up an ASP.NET Core web application and publish it to Azure.

Do these:

Research the C# *DateTime.Now* property (i.e., Google the term, read articles, watch videos, etc.).

HELPFUL LINK:

[DateTime.Now Property \(System\)](#)

Research *event handler*.

HELPFUL LINKS:

[What is an Event \(Event Handler\)?](#)

[Handling and Raising Events](#)

Research *handler method* as it relates to Razor Pages. Also, research these methods: *OnPost()* and *OnGet()*.

HELPFUL LINKS:

[Handler Methods in Razor Pages](#)

[Razor Pages - Understanding Handler Methods](#)

Research *void* (as it relates to C#).

HELPFUL LINKS:

[C# Methods](#)

[void - C# reference](#)

Create a new ASP.NET Core web application project.

HELPFUL LINK:

[Create an ASP.NET Core web app in Azure - Create](#)

Design a simple ASP.NET Core web app that displays the actual server time in the browser.

HELPFUL LINK:

[Step 2: Creating Your First ASP.NET Core Web App - Visual Studio](#)

Customize the CSHTML <h2> element with text of your choosing.

HELPFUL LINK:

[ASP.NET Razor C# Syntax](#)

Publish your ASP.NET Core web application to Azure.

IMPORTANT NOTE: Make sure to select a **free** account. You do not need/want to pay for any Azure services. To avoid charges, be sure to choose a free account.

HELPFUL LINK:

[Publish your ASP.NET app to Azure - Publish](#)

Using the skills gained on this course and through online research (self study and debugging), perform these actions:

Research *Code-First* (i.e., Google the term, read articles, watch videos, etc.).

HELPFUL LINKS:

[What is Code First?](#)

[Code First to a New Database](#)

Create a basic Entity Framework Code-First console application that creates a basic Student database and adds one student.

IMPORTANT NOTE: If you use the below HELPFUL LINK, do not continue past “Note: If you modify these classes and run the application again, then it will throw the following exception.” The steps that follow this webpage (such as “Code-First Conventions” and “DB Initialization”) are not required for the purposes of this challenge. All that is required is the code given on the webpage provided below.

HELPFUL LINK:

[Entity Framework Code First](#)

Upload your code to GitHub.



C# Projects

These are some small projects I made using C# and .NET Framework

Projects:

- [Car Insurance](#)
- [NFL Stats](#)

Car Insurance

This project is an MVC web application that mimics a car insurance website. It takes user's input on policy information and calculates a quote based on business logic. There is also an admin page that allows staff to view all of the quotes that have been issued.

NFL Stats

This is a data visualization application that is meant to store and display football stats for the purposes of fantasy football. It is an MVC web application that utilizes Entity Framework 6 for data access and a Code First workflow. It is still a work in progress.